

SC1006: Introduction to Computer Organisation

Lecture Notes — Chapters 1–8

2020

Contents

1	Introduction	5
1.1	What is a Computer?	5
1.2	Classes of Computers	5
1.2.1	Supercomputers	5
1.2.2	Microcomputers	5
1.2.3	Embedded Systems	5
1.3	Early Days of the Digital Computer	6
1.3.1	Harvard and Von Neumann Architectures	6
1.3.2	ENIAC — The First Digital Computer	6
1.4	Basic Components of a Microcomputer	7
1.4.1	Core Components and the Bus	7
1.4.2	The CPU Clock	7
1.4.3	Reset Circuitry	8
1.5	Desktop PC and Tablet PC — Hardware Composition	8
1.5.1	Inside a Desktop PC	8
1.5.2	Inside a Tablet Computer — iPad 2 Case Study	8
1.5.3	Apple A5 — A System-on-a-Chip (SoC)	9
2	Data Organisation in Memory	10
2.1	Role of Memory in Computing	10
2.1.1	Programming in Hardware vs. Software	10
2.1.2	Code, Data, and Memory	10
2.1.3	The Von Neumann Stored-Program Concept	10
2.1.4	The Memory Hierarchy	11
2.1.5	Characteristics of Main Memory	11
2.2	Number Representation	11
2.2.1	Basic Numeric Types in C	11
2.2.2	Signed and Unsigned Integers	12
2.2.3	ANSI C Numeric Data Types	12
2.2.4	Multi-byte Storage and Endianness	12
2.3	Character and Boolean Representation	13
2.3.1	Character Representation	13
2.3.2	Boolean Representation	14
2.4	Array, String, and Structure Representations	14
2.4.1	Arrays	14
2.4.2	Strings	14
2.4.3	Structures	15

2.5	Pointer Representation	15
2.6	Data Alignment	16
2.6.1	Alignment Requirements	16
2.6.2	Why Alignment Matters: Intel Pentium 4 Case Study	16
2.6.3	Data Padding in Structures	16
3	Instruction Organisation in Memory	18
3.1	Characteristics of Instructions and the ARM Programmer's Model	18
3.1.1	Code and Data in Memory	18
3.1.2	Instruction Format: Opcode and Operands	18
3.1.3	Addressing Modes	18
3.1.4	Role and Categories of Instructions	19
3.1.5	The ARM MOV Instruction	19
3.1.6	The ARM Programmer's Model	20
3.1.7	Program Counter (PC / R15)	20
3.1.8	Stack Pointer (SP / R13) and Link Register (LR / R14)	20
3.1.9	CPSR Condition Code Flags	20
3.2	Basic Execution Cycle	21
3.2.1	The LDR Instruction	21
3.2.2	The Role of the Processor and the Fetch–Decode–Execute Cycle	21
3.2.3	A Simple Processor: Internal Structure	22
3.2.4	Execution of LDR R1, [R0]: Step-by-Step	22
3.2.5	Performance Considerations	23
4	Addressing Modes	24
4.1	Introduction to Assembly Programming	24
4.1.1	Why Use Assembly Language?	24
4.1.2	When to Use Assembly Language?	24
4.2	Addressing Modes — Overview	25
4.3	Register Direct Addressing	25
4.4	Immediate Addressing	26
4.4.1	Encoding of the 12-bit Immediate Field	26
4.5	Register Indirect Addressing	27
4.5.1	Motivation	27
4.5.2	The LDR Instruction	27
4.5.3	The STR Instruction	27
4.5.4	Data Alignment Constraint	27
4.5.5	Register Indirect with Offset (Base + Offset)	27
4.5.6	Register Indirect with Index Register (Base + Index)	28
4.6	Register Indirect with Autoindexing	28
4.6.1	Concept	28
4.6.2	Pre-index vs. Post-index	28
4.7	The System Stack and Stack Implementations	29
4.7.1	Full Descending (FD) Stack — Default ARM Stack	29
4.7.2	Empty Ascending (EA) Stack — Alternative Implementation	30
4.8	PC-related Addressing Modes	30
4.8.1	Absolute vs. Relative Jumps	30
4.8.2	Position-Independent Code	30
4.8.3	PC-Relative Data Addressing	30

5	Instruction Set	32
5.1	Data Transfer Instructions	32
5.1.1	Register Data Transfer: MOV and MVN	32
5.1.2	Memory Data Transfer: LDR and STR	32
5.1.3	Program Example: Copying a Block of Memory	33
5.2	Arithmetic Instructions	34
5.2.1	ADD Instruction	34
5.2.2	SUB and RSB Instructions	35
5.2.3	Carry-Based Arithmetic: ADC, SBC, RSC	35
5.3	Logical, Shift, and Rotate Instructions	35
5.3.1	Logical Instructions: AND, ORR, EOR, MVN	35
5.3.2	Shift and Rotate Operations	36
5.4	Program Control Instructions	37
5.4.1	Conditional Branch: Bcc	37
5.4.2	Conditional Test Instructions: CMP, CMN, TST, TEQ	38
5.4.3	Conditional Execution	38
5.5	Program Example: Finding the Largest Number (FindMax)	39
6	Modular Programming	40
6.1	Motivation: Modular Program Design	40
6.2	Subroutines	41
6.2.1	Why Not Use Plain B?	41
6.2.2	Branch with Link (BL)	41
6.2.3	Returning: BX LR and MOV PC, LR	41
6.3	Parameter Passing	42
6.3.1	Parameter Passing via Registers	42
6.3.2	Parameter Passing via Memory Block	42
6.4	Examples	43
6.4.1	Bit-Counting Subroutine (Passing via Registers)	43
6.4.2	Lower-to-Upper-Case Subroutine (Passing via Memory)	43
6.5	The System Stack	44
6.5.1	Push and Pop Instructions	44
6.6	Parameter Passing via the Stack	45
6.7	Transparent Subroutines (Revisited)	46
6.8	Passing by Value and by Reference	46
6.9	Local Variables and the Stack Frame	47
6.9.1	Accessing the Stack Frame via the Frame Pointer	47
6.9.2	Accessing the Stack Frame via the Stack Pointer	48
6.10	Nested Subroutines	48
6.10.1	The LR Overwrite Problem	48
6.10.2	Solution: Saving LR onto the System Stack	48
6.10.3	Worked Example: Dot-Product with Nested Multiply	49
6.11	Recursive Subroutines	50
6.11.1	Requirements for Correct Recursion	50
6.11.2	Worked Example: Fibonacci Sequence	51
6.12	Chapter Summary	52
7	Flow-Control Constructs	53
7.1	IF Statements	53
7.1.1	Example: Find Largest Number	53
7.2	Conditional Execution in ARM	54
7.2.1	Find Largest with Conditional Execution	54

7.2.2	Find Largest Number and Store Its Index	55
7.3	IF-ELSE Statements	55
7.3.1	Conditional Execution for IF-ELSE	56
7.4	Compound Conditions	56
7.4.1	Compound AND Conditions	56
7.4.2	Compound OR Conditions	56
7.5	Branchless Logic	57
7.6	SWITCH Statements	58
7.6.1	Consecutive Narrow-Range Values — Jump Table	58
7.6.2	Random Wide-Range Values — Forked if-else-if	59
7.6.3	Extended Example: Grade Statistics Subroutine	59
7.7	Loop Constructs	61
7.7.1	WHILE Implementation	61
7.7.2	DO-WHILE Implementation	61
7.7.3	FOR Implementation	61
7.7.4	Example: Summation 1 to N	62
7.8	Chapter Summary	62
8	Instruction Encoding and ISA	64
8.1	Introduction	64
8.2	Instruction Encoding Fundamentals	64
8.2.1	What is Encoded in an Instruction?	64
8.3	ARM Instruction Format	64
8.3.1	Arithmetic and Logic Instructions	65
8.3.2	Comparison Instructions	66
8.3.3	MOV Instructions	66
8.3.4	Shift and Rotate Instructions	66
8.3.5	Branch Instructions	67
8.3.6	Load/Store Instructions	67
8.3.7	Load/Store Multiple Instructions	68
8.4	Instruction Type and Condition Field Encoding	68
8.4.1	Instruction Type Field	68
8.4.2	Condition Field	68
8.5	Fixed-Length vs. Variable-Length ISA	69
8.5.1	Fixed-Length ISA	69
8.5.2	Variable-Length ISA	69
8.6	Design Considerations in ISA	70
8.6.1	Using Registers Efficiently	70
8.6.2	Implications of Register Count	71
8.6.3	Orthogonality of ISA	71
8.7	Why Good ISA Design Matters	72
8.7.1	Custom Processor Design in Industry	72
8.8	Chapter Summary	73

Chapter 1

Introduction

1.1 What is a Computer?

A computer is any device that contains some form of computational elements. The range of such devices is enormous — from room-sized supercomputers all the way down to wristwatches. There is no single definitive classification scheme, but three broad categories are commonly recognised: **supercomputers**, **microcomputers**, and **embedded systems**.

1.2 Classes of Computers

1.2.1 Supercomputers

Supercomputers are very large, powerful, and expensive machines designed for tasks that demand extremely high computational throughput and the ability to operate on very large datasets (e.g. high-precision calculations). They are generally scalable by adding more processors.

Example 1.2.1 (Titan Supercomputer). The Titan supercomputer at Oak Ridge National Laboratory, USA, achieved a computational peak performance of approximately 17–27 petaFLOPS. It comprised more than 18,000 Nvidia Tesla K20 GPUs and 700 terabytes of memory.

Typical application domains include weather forecasting, simulation of complex physical systems, and modelling of sub-atomic structures (e.g. assessing the structural health of nuclear weapons).

1.2.2 Microcomputers

Microcomputers use a **microprocessor** as their central processing unit and rely on external memory and peripheral chip support. They range from high-end server workstations down to desktop PCs and notebooks used for everyday home-office computing.

1.2.3 Embedded Systems

Embedded systems are compact devices that typically employ a **single-chip microcontroller** containing the processing unit, memory, and all relevant peripheral support on one integrated circuit. They are called “embedded” because the presence of a microprocessor inside the device is non-obvious to the user. Examples include digital cameras, thermometers, portable music players, and computer mice.

Key Result: Three Classes of Computers

- **Supercomputer** — massively parallel, petaFLOP-scale performance, used for scientific simulation.
- **Microcomputer** — microprocessor + external memory; covers servers, desktops, and laptops.
- **Embedded system** — single-chip microcontroller; the processor is hidden inside the product.

1.3 Early Days of the Digital Computer

Major advances in computing occurred during World War II (the 1940s), when computer research was funded primarily by the War Department to solve problems related to ballistics. Early fire-control systems were *analog*, gear-based computers: operators dialled in parameters such as target speed and course, range to target, wind speed and direction, and own vessel speed and course, and the mechanical outputs then drove the motors of the gun.

1.3.1 Harvard and Von Neumann Architectures

From this era, two major computer architecture designs emerged.

Definition 1.3.1 (Harvard Architecture). The **Harvard architecture**, named after the Harvard series of relay calculators developed by Howard Aiken at Harvard University (commercialised as IBM's ASCC, also known as the Harvard Mark 1), uses *two separate memory spaces*: one for program code and one for data. The CPU can therefore fetch an instruction and read/write data simultaneously.

Definition 1.3.2 (Von Neumann Architecture). The **Von Neumann architecture**, developed by John von Neumann at Princeton University, uses a *single shared memory* for both program code and data. The CPU reads instructions and data from the same memory bus.

A high-level comparison of the two architectures is given below:

Property	Harvard	Von Neumann
Memory spaces	Two (code + data)	One (shared)
Simultaneous fetch	Yes	No (von Neumann bottleneck)
Hardware cost	Higher	Lower
Prevalence today	Microcontrollers, DSPs	General-purpose CPUs

Exam Tip: Identifying the Architectures

Remember: **Harvard** $\Rightarrow$ **H**ard to confuse — it has two Halves of memory. Von Neumann has **one** shared memory space, which introduces the “von Neumann bottleneck” (CPU waits for memory access). In practice, most modern CPUs use a modified Harvard design internally (separate instruction and data caches) but present a von Neumann interface to software.

1.3.2 ENIAC — The First Digital Computer

In 1943 the US Army funded Presper Eckert and John Mauchly at the University of Pennsylvania to construct ENIAC (**E**lectronic **N**umerical **I**ntegrator **A**nd **C**alculator), based on von Neumann's architecture. ENIAC's physical specifications illustrate how far hardware has come:

- Weight: 30 tons
- Vacuum tubes: 19,000
- Relays: 1,500
- Power consumption: 200 kW
- Main memory: only 1.5 kbits

The high cost of memory in the early days of computing made the shared-memory approach of the von Neumann design preferable, and it has remained the dominant architecture for general-purpose computers ever since.

1.4 Basic Components of a Microcomputer

1.4.1 Core Components and the Bus

A microcomputer consists of three main components:

1. **Processor** (CPU) — executes instructions.
2. **Main Memory** — stores both data and the running program (von Neumann model).
3. **I/O Interfaces** — connect input devices (keyboard, sensors) and output devices (display, actuators) to the system.

These components are interconnected by a **bus**, which is a collection of parallel wires over which binary information is transferred simultaneously. The bus is logically divided into three sub-buses:

- **Data bus** — carries the actual data values being transferred.
- **Address bus** — specifies the memory location or I/O port being accessed.
- **Control bus** — carries timing and control signals (read, write, interrupt, etc.).

Other important support components include a **power supply unit**, a **CPU clock circuit**, and **reset circuitry**.

1.4.2 The CPU Clock

Most computers are **synchronous**: all operations are coordinated by a master (system) clock signal. Key points:

- The speed performance of the computer is governed by the **frequency** of the clock.
- The CPU requires a fixed number of clock ticks (**cycles**) to execute each instruction.
- Many different clock frequencies are *derived* from the one master clock oscillator.
- Operations close to the CPU core (registers, ALU) are clocked **faster**; operations involving external components (main memory, peripheral access) are clocked **slower**.

Warning: Clock Speed $\neq$ Performance

A fast processor clock does *not* guarantee fast execution if memory is slow. A CPU with a fast clock coupled to slow main memory will still execute instructions slowly, because it must stall waiting for data from memory. Memory access speed is often the true performance bottleneck.

1.4.3 Reset Circuitry

The reset circuit ensures the CPU starts in a well-defined, known state. On power-up, the reset circuitry asserts an active-low signal on the CPU's **RESET** pin for several clock cycles, after which the CPU begins executing its boot-up code from a fixed address. Most systems also provide a **manual reset button** that can re-assert the reset signal without cutting power.

1.5 Desktop PC and Tablet PC — Hardware Composition

Despite differences in form factor, both a desktop PC and a tablet computer instantiate the same abstract model: CPU, memory, and I/O interfaces. The difference lies in physical packaging and power constraints.

1.5.1 Inside a Desktop PC

The major hardware components of a typical desktop PC are:

- **Motherboard with CPU** — the main circuit board integrating the processor socket, chipset, and peripheral connectors.
- **DRAM Main Memory** — typically 4–8 GiB of dynamic RAM.
- **Hard Disk Drive (HDD)** — secondary (non-volatile) storage, e.g. 1 TiB.
- **Graphics Processing Unit (GPU)** — a co-processor dedicated to graphics rendering and, increasingly, general parallel computation.
- **Power Supply Unit (PSU)** — converts mains AC to the DC voltages required by the components.
- **I/O Peripherals** — display, keyboard, mouse, etc.

1.5.2 Inside a Tablet Computer — iPad 2 Case Study

The iPad 2 illustrates how a tablet consolidates these same building blocks into a compact, battery-powered form:

- **LCD display** + front glass panel (touchscreen).
- **Circuit board** hosting all silicon.
- **3-cell Li-Ion polymer battery** (25 Wh, providing ≈ 10 hours of operation).
- **Back chassis** (structural enclosure).

On the circuit board itself:

- **A5 processor** (CPU + GPU SoC).
- **Flash NAND** — 16 GiB secondary (non-volatile) memory.
- **Touchscreen controller and driver ICs**.
- **LCD timing controller**.
- **Power Management IC**.

1.5.3 Apple A5 — A System-on-a-Chip (SoC)

The A5 is a **Package-on-Package (PoP) System-on-a-Chip (SoC)** designed by Apple and manufactured by Samsung. Its specifications:

- **CPU:** Dual-core ARM Cortex-A9 at 1 GHz (clock speed can be dynamically reduced to save battery), with 4.5 MiB cache.
- **GPU:** Dual-core PowerVR SGX543MP2 (accelerates graphics).
- **RAM:** 512 MiB low-power DDR SDRAM at 533 MHz.

Definition 1.5.1 (Package-on-Package (PoP)). **Package-on-Package** is an IC packaging technique that vertically stacks and interconnects separate packages (e.g. a CPU package and a memory package) via Ball Grid Array (BGA) solder connections.

The advantages of PoP packaging are:

1. **Space saving** — stacking reduces the motherboard footprint, enabling thinner/smaller products.
2. **Minimised track length** between CPU and memory — faster signal propagation and reduced electrical noise.
3. **Independent testability** — memory and CPU packages can be tested separately, improving manufacturing yield and allowing multiple memory suppliers.
4. **Flexible memory sizing** — different memory capacities can be paired with the same CPU die, simplifying inventory management.

Key Result: Same Abstraction, Different Packaging

Whether it is a desktop tower, a tablet, or a wristwatch, the fundamental architecture is identical: **CPU + Memory + I/O**. Form factor, power budget, and packaging technology (e.g. PoP SoC vs. discrete motherboard components) differ, but the logical model does not.

Chapter 2

Data Organisation in Memory

2.1 Role of Memory in Computing

2.1.1 Programming in Hardware vs. Software

There are two fundamental approaches to making a processor perform a computation.

Programming in hardware wires up a fixed sequence of arithmetic and logic functions tailored to a specific task. For example, computing $A = (B + C) \times D$ with hardware programming would require a dedicated adder feeding into a dedicated multiplier. This approach is extremely fast because there is no decoding overhead, but it is entirely inflexible — changing the algorithm means redesigning the circuit.

Programming in software uses a single general-purpose arithmetic and logic unit (ALU) controlled by an *instruction decoder*. Instruction codes fetched from memory tell the decoder which operation to activate on the ALU. Although slower and more complex than pure hardware, this approach allows any computation to be specified by writing a different sequence of instructions, with no changes to the physical hardware.

Key Result: Why Software Wins

Software programming trades a small speed penalty for enormous flexibility. The **instruction decoder** is the key component: it interprets bit patterns (opcodes) and activates the corresponding function in a general-purpose ALU, enabling any algorithm to run on the same physical hardware.

2.1.2 Code, Data, and Memory

Two fundamental concepts underlie every program:

Definition 2.1.1 (Code and Data). **Code** is a sequence of instructions. **Data** are the values those instructions operate on (inputs and intermediate results). In a stored-program computer both reside in the same memory.

Definition 2.1.2 (Memory). **Memory** is a sequential list of addressable storage locations. Each location holds a fixed-width binary value. A processor selects a location by driving its binary address onto the *address bus*; the contents of that location then appear on the *data bus*.

2.1.3 The Von Neumann Stored-Program Concept

Recalling Definition 1.3.2 from Chapter 1, most modern computers follow von Neumann's stored-program concept, which rests on three principles:

1. Both data and instructions are stored in the **same memory**.
2. Memory contents are addressable by **location**, without regard to the type of data stored there.

3. Execution proceeds **sequentially** (one instruction after another) unless explicitly altered by a branch or jump instruction.

2.1.4 The Memory Hierarchy

Real systems cannot be built with a single memory technology that is simultaneously fast, large, and cheap. Instead, memories are organised in a *hierarchy* of levels, ordered from fastest/most expensive to slowest/cheapest:

Level	Technology	Access Time	Typical Size
Registers	Flip-flops (in CPU)	1 clock cycle	2–128 registers
Cache memory	Static RAM (SRAM)	3–20 ns	up to 512 kB
Main memory	Dynamic RAM / ROM	30–70 ns	up to 16 GB
Secondary memory	Magnetic / Flash	0.03–100 ms	up to 4 TB

As one moves down the hierarchy, cost per bit decreases, capacity increases, and access time increases. The CPU accesses levels in order — if data is not found in a fast level, it fetches it from the next level down (a *cache miss*).

2.1.5 Characteristics of Main Memory

Main memory is organised as a flat array of byte-sized (8-bit) locations, each with a unique **address**:

- Each address is specified as a binary pattern driven onto the address bus.
- Memory size = 2^n bytes, where n is the number of address-bus lines.
- Both data and instructions are stored in consecutive locations.
- Access is **random**: any location can be read or written in the same time, regardless of its address.

Exam Tip: Memory Size Formula

If an address bus has n lines, the maximum addressable memory is 2^n bytes. For example, a 16-bit address bus can address $2^{16} = 65,536$ bytes (64 kB); a 32-bit bus can address $2^{32} \approx 4$ GB.

2.2 Number Representation

2.2.1 Basic Numeric Types in C

Programming languages expose the hardware representation of numbers through *data types*. In ANSI C, numbers come in two basic varieties:

- **Integers** (`int`) — whole numbers, e.g. 32,676.
- **Floating-point numbers** (`float`) — numbers with a fractional part, e.g. 3.2676×10^4 . Useful for scientific calculation; always signed. Floating-point arithmetic will be covered in detail in the Computer Arithmetic chapter.

Some processors handle only integer arithmetic natively and use a separate **Floating-Point Unit (FPU)** co-processor for `float` operations.

2.2.2 Signed and Unsigned Integers

Integers can be either **signed** (positive and negative) or **unsigned** (non-negative only):

- `int` — signed integer, e.g. `-1`.
- `unsigned int` — unsigned integer, e.g. `255`.

Most processors represent signed integers using the **two's complement** scheme. For an n -bit value, the range is -2^{n-1} to $2^{n-1} - 1$ (signed) vs. 0 to $2^n - 1$ (unsigned). For example, with 8 bits:

Bit Pattern	Two's Complement (signed)	Unsigned
0111 1111	+127	127
1111 1111	-1	255

Warning: Signed vs. Unsigned Interpretation

The *same bit pattern* in memory has a different numeric meaning depending on whether it is interpreted as signed or unsigned. `1111 1111` is `-1` in two's complement but `255` as an unsigned byte. Always declare variables with the correct signedness to avoid subtle bugs.

Use `unsigned` where only non-negative values are needed (e.g. counting a population) to extend the positive range.

2.2.3 ANSI C Numeric Data Types

The range of a numeric type is determined by its size in bits. The table below summarises the standard ANSI C numeric types (sizes may vary with processor and compiler):

Type	Bytes	Bits	Range
<code>signed char</code>	1	8	-128 to $+127$
<code>unsigned char</code>	1	8	0 to $+255$
<code>short int</code>	2	16	$-32,768$ to $+32,767$
<code>unsigned short int</code>	2	16	0 to $+65,535$
<code>int</code>	4	32	$-2,147,483,648$ to $+2,147,483,647$
<code>unsigned int</code>	4	32	0 to $+4,294,967,295$
<code>long long int</code>	8	64	$-(2^{63})$ to $2^{63} - 1$
<code>float</code>	4	32	(IEEE 754 single precision)
<code>double</code>	8	64	(IEEE 754 double precision)

Use the **smallest** type whose range covers the required values in order to minimise memory usage.

2.2.4 Multi-byte Storage and Endianness

When a multi-byte integer (e.g. a 32-bit `int`) is stored in memory, the four individual bytes must be laid out in consecutive addresses. There are two conventions:

Definition 2.2.1 (Big-Endian). In **big-endian** byte ordering the *most significant byte* is stored at the *lowest* address. Used by, e.g., Motorola 68k, Sun SPARC, Mac PowerPC.

Definition 2.2.2 (Little-Endian). In **little-endian** byte ordering the *least significant byte* is stored at the *lowest* address. Used by, e.g., Intel x86, DEC Alpha, Intel 8051.

For a 32-bit value stored at address N :

Address	Big-Endian	Little-Endian
N	MSB	LSB
$N + 1$	...	...
$N + 2$	...	...
$N + 3$	LSB	MSB

Exam Tip: Endianness Quick Check

To determine the endianness: look at the *lowest* address in memory. If it holds the *most* significant byte, it is **big-endian** (big end first). If it holds the *least* significant byte, it is **little-endian**. The Intel x86 family (and most modern PCs) is little-endian.

2.3 Character and Boolean Representation

2.3.1 Character Representation

Computers do not only process numbers; many applications manipulate text. In ANSI C a single character is declared as `char` and occupies exactly **one byte** of memory.

Raw bytes in memory are transformed into human-readable characters via an **encoding standard** that maps each integer code to a symbol.

ASCII. The **American Standard Code for Information Interchange (ASCII)** is the most widely used 7-bit character encoding. Key properties:

- Codes 0–31 and 127 are *control characters* (non-printable), e.g. 0x0A = Line Feed, 0x0D = Carriage Return.
- Codes 32–126 are printable characters.
- Uppercase letters (A–Z): 0x41–0x5A.
- Lowercase letters (a–z): 0x61–0x7A.
- Decimal digits (0–9): 0x30–0x39.

The fact that all uppercase letters, lowercase letters, and digits occupy *contiguous ranges* makes it easy to check a character's category or convert between upper and lower case using simple arithmetic.

A byte can store an ASCII character in its lower 7 bits; the most significant bit (MSB) can be used for **parity error checking**.

SIXBIT. DEC's SIXBIT encoding, popular with PDP-8 and PDP-10 computers in the 1960s–70s, encodes 64 symbols in 6 bits. It lacks control characters (no CR/LF) and is therefore unsuitable for general text processing, but it allowed three characters to be packed into an 18-bit PDP word.

Unicode. To represent all major living languages, the **Unicode** standard uses 8-, 16-, or 32-bit code points and is downward compatible with ASCII. Unicode 13.0 (released March 2020) contains 143,859 characters, including CJK ideographs, emoji, and historical scripts. It is the encoding underlying XML, Java, C#/.NET, and most modern operating systems.

2.3.2 Boolean Representation

A Boolean variable has exactly two states: **true** and **false**. In ANSI C (C99 and later) the Boolean type is `_Bool`, accessible via `#include <stdbool.h>`. Semantics:

- `false` = 0
- `true` = any non-zero value (conventionally 1)

Memory storage for Boolean variables is **inefficient**: although a Boolean requires only 1 bit of information, most implementations allocate a full byte (the minimum addressable unit) per variable.

Example 2.3.1 (Bit-Addressable Memory — Intel 8051). The Intel 8051 microcontroller partially addresses this inefficiency by providing 128 **bit-addressable** memory locations (bytes 0x20–0x2F of internal RAM). Each of these 16 bytes can be accessed either as a full byte or as 8 individually addressable bits, giving 128 independently addressable flags. This is ideal for embedded applications where many Boolean status signals (switch states, LED flags) must be tracked efficiently.

2.4 Array, String, and Structure Representations

2.4.1 Arrays

Definition 2.4.1 (Array). A **linear array** is a contiguous block of memory storing a sequence of values of the *same* data type. Elements are accessed via an offset from the **base address** (BA) of the array.

For an array `a` of elements of type T (each of size s bytes), the address of element `a[n]` is:

$$\text{Address}(\mathbf{a}[\mathbf{n}]) = \text{BA} + n \times s$$

Example 2.4.2 (Char and Short-Int Arrays). Consider the C declarations:

```
1 char c[2];           // 1-byte elements: c[0] at BA_c, c[1] at BA_c + 1
2 short int i[3];      // 2-byte elements: i[n] at BA_i + 2*n
3 i[0] = 5; i[1] = 6; i[2] = 7;
```

Element `i[2]` is stored at $\text{BA}_i + 4$, occupying two consecutive bytes 0x00, 0x07 (big-endian) or 0x07, 0x00 (little-endian).

Nested (multi-dimensional) arrays. Each element of an array may itself be an array. For a 2-D array `int k[R][C]`, the offset of element `k[a][b]` from the base address is:

$$\text{Offset}(\mathbf{k}[\mathbf{a}][\mathbf{b}]) = \text{sizeof}(\text{int}) \times (C \cdot a + b)$$

For example, `k[2][1]` in a `k[3][2]` array of `ints` (4 bytes each) has offset $4 \times (2 \times 2 + 1) = 20$ bytes.

2.4.2 Strings

A **string** in C is an array of `char` values. Two storage conventions exist:

Definition 2.4.3 (C String). A **C string** is a character array terminated by a **null character** (0x00). The end of the string is found by scanning forward until the null terminator is encountered. The string "123" requires 4 bytes: three character bytes (0x31, 0x32, 0x33) plus the null byte.

Definition 2.4.4 (Pascal String). A **Pascal string** stores the **length** of the string in the first byte, followed by the character bytes (no null terminator). The string "123" requires 4 bytes: 0x03 (length) then 0x31, 0x32, 0x33.

Property	C string	Pascal string
Length retrieval	$O(n)$ scan	$O(1)$ read first byte
Maximum length	Unlimited	255 characters (1-byte length)
Language support	C, C++	Pascal, older Mac APIs

2.4.3 Structures

Definition 2.4.5 (Structure). A **structure** (**struct** in C) is a composite data type that groups fields of *different* types into a single named unit. When a structure variable is declared, the compiler allocates consecutive memory for all its fields in declaration order.

Example 2.4.6 (Structure Memory Layout).

```

1 struct Man {
2     char race;    // 1 byte at BA_m
3     int age;      // 4 bytes at BA_m + 1 (without padding)
4     char name[20]; // 20 bytes at BA_m + 5
5 };
6 struct Man m;
```

Field `m.age` is at offset 1; field `m.name` is at offset 5 (immediately after the 4-byte `int`).

The key difference between arrays and structures: arrays hold *homogeneous* data; structures hold *heterogeneous* data. Memory space is allocated only when a variable of the structure type is declared (not when the type itself is defined).

2.5 Pointer Representation

Definition 2.5.1 (Pointer). A **pointer** is a variable whose value is a **memory address**. In C, a pointer is declared by prefixing the base type with `*`, e.g. `char *ptr` declares a pointer to a `char`.

Key properties of pointers:

- The **size** of a pointer is determined by the processor's address space, *not* by the data type it points to. On a processor with 16-bit addresses, all pointers are 2 bytes; on a 64-bit system, all pointers are 8 bytes.
- The **value** of a pointer is obtained using the address-of operator `&`: e.g. `ptr = &c` stores the address of variable `c` in `ptr`.
- The **dereferencing operator** `*` reads or writes the memory location the pointer points to: `*ptr = 'Z'` writes 'Z' to the address held in `ptr`.

Example 2.5.2 (Pointer in Memory (16-bit address space)).

```

1 char c;           // 1 byte at 0x0100
2 char *ptr;        // 2 bytes (16-bit address)
3 c = 'A';
4 ptr = &c;         // ptr contains 0x0100
5 *ptr = 'Z';       // c is now 'Z'
```

After execution: address 0x0100 holds 0x5A ('Z'); the pointer variable `ptr` itself holds the 16-bit value 0x0100.

Pointers are especially powerful for navigating arrays and structures: knowing the base address of an array or struct, one can access any element by adding the appropriate byte offset.

2.6 Data Alignment

2.6.1 Alignment Requirements

Most modern processors impose restrictions on the allowable *starting address* for multi-byte data types: the address must be a **multiple of the data type's size**. This is called **data alignment**.

For programs compiled with Visual C++ or GCC targeting a 64-bit Intel processor:

Type	Size (bytes)	Allowable start addresses
char	1	any address
short	2	multiples of 2: 0x0000, 0x0002, ...
int	4	multiples of 4: 0x0000, 0x0004, ...
float	4	multiples of 4
double	8	multiples of 8: 0x0000, 0x0008, ...
pointer	8	multiples of 8

2.6.2 Why Alignment Matters: Intel Pentium 4 Case Study

The Pentium 4 has a 64-bit (8-byte) data bus backed by eight 8-bit memory modules. Only one address can exist on the address bus at a time. An 8-byte value that *spans two 8-byte boundaries* therefore requires **two memory cycles** to fetch, halving throughput for that access. An aligned value crosses no boundary and requires only **one cycle**.

The three lowest address bits (A_2--A_0) are replaced by byte-enable lines BE0--BE7 that select which of the eight byte lanes are active; the actual byte address is thus determined by $A_{35}--A_3$ together with the byte-enables. A misaligned transfer would require A_3 to be in two different states simultaneously — which is impossible in one cycle.

Warning: Misaligned Access is Slow

On processors that permit misaligned accesses (e.g. x86), the hardware silently issues two memory transactions. This can halve memory throughput for that datum. Some processors (e.g. early MIPS, SPARC) raise a hardware exception on misaligned access. Always align data to the natural boundary of its type.

Effect of bus width. An 8-bit-bus processor (e.g. Intel 8080) transfers one byte per cycle regardless of address, so it has *no* alignment requirements. A 16-bit-bus processor requires 2- and larger data types to start at *even* addresses.

2.6.3 Data Padding in Structures

When fields of different sizes are packed into a **struct**, the compiler inserts **padding bytes** between fields (and at the end of the struct) to satisfy alignment constraints.

Example 2.6.1 (Structure Padding).

```

1 struct rec1 {
2     char c;        // 1 byte at offset 0
3     int i;         // needs 4-byte alignment -> 3 padding bytes at offsets 1-3
4                     // i starts at offset 4
5     char d[2];     // 2 bytes at offsets 8-9
6                     // 2 padding bytes at offsets 10-11 (to round struct to 4 bytes)
7 };
8 // sizeof(struct rec1) = 12 bytes (with padding)

```

Minimising padding by reordering fields. Declaring fields in **decreasing size order** reduces wasted padding:

Example 2.6.2 (Reordered Structure).

```

1 struct rec2 {
2     int i;         // 4 bytes at offset 0
3     char c;        // 1 byte at offset 4
4     char d[2];     // 2 bytes at offsets 5-6
5                     // 1 padding byte at offset 7
6 };
7 // sizeof(struct rec2) = 8 bytes -- saves 4 bytes vs rec1

```

Exam Tip: Minimising Structure Size

Declare structure members in **descending size order** (largest type first, smallest last). This reduces the amount of padding the compiler inserts and can significantly shrink the struct, especially in large arrays of structures.

Key Result: Data Alignment Summary

- A datum of size s bytes must start at an address that is a **multiple of s** .
- Compilers insert **padding bytes** to enforce alignment within structures.
- **Reorder fields** from largest to smallest to minimise padding.
- The strictness of alignment depends on the width of the processor's data bus.

Chapter 3

Instruction Organisation in Memory

3.1 Characteristics of Instructions and the ARM Programmer's Model

3.1.1 Code and Data in Memory

Recall from Chapter 2 that both program code and data are stored together in the same memory (Definition 2.1.1). Instructions are stored as binary patterns called **machine codes**; they are also represented in human-readable **mnemonics**. Memory is partitioned into separate **code** and **data** segments.

As a concrete example, the ARM processor is a 32-bit CPU in which every instruction is exactly 32 bits (4 bytes) long. The two-instruction sequence

```
1 MOV R1, R0
2 MOV R2, R1
```

is stored in code memory as four consecutive bytes per instruction (e.g. 0xE1A00100 for MOV R2,R1), while data values are stored in a separate region at a higher address range.

3.1.2 Instruction Format: Opcode and Operands

Definition 3.1.1 (Instruction Format). Most instruction formats consist of **two parts**:

- **Op-code** — a bit field at the start of the instruction word that specifies the operation to be carried out (e.g. move, add, subtract).
- **Operands** — the remaining bit fields that specify the data itself, or the location of the data, and where the result is to be stored.

Some operations produce no storable result but instead alter the program sequence or influence the processor status flags (e.g. the flags N, Z, V, C).

Encoding the opcode. The number of bits required for the opcode is determined by the number of distinct operations the instruction set must encode. If there are k different operations, the opcode field must be at least $\lceil \log_2 k \rceil$ bits wide. For example, with 80 different operations, since $2^6 = 64 < 80 \leq 128 = 2^7$, a minimum of **7 bits** is required. The greater the variety of operations the instruction set supports, the longer the instruction word must be.

Key Result: Instruction = Opcode + Operands

Every instruction word encodes two things: *what* to do (opcode) and *what to do it to* (operands). The minimum opcode width for k operations is $\lceil \log_2 k \rceil$ bits.

3.1.3 Addressing Modes

An **addressing mode** is the method by which an operand's location is specified within an instruction (see Definition 4.2.1 in Chapter 4 for a full treatment). Common modes include

immediate, register direct, register indirect, and variants that add a constant offset or index register to a base address.

The table below summarises the principal addressing modes with examples from both the ARM and Intel instruction sets:

Mode	ARM example	Intel example
Absolute (Direct)	(none in ARM)	MOV AX, [1000h]
Register Direct	MOV R1, R0	MOV AX, DX
Immediate	MOV R1, #3	MOV AX, 0003h
Register Indirect	LDR R1, [R0]	MOV AX, [BX]
Register Indirect with Offset	LDR R1, [R0, #4]	MOV AX, [BX+4]
Register Indirect with Index	LDR R1, [R0, R2]	MOV AH, [BX+DI]
Implied	BNE LOOP	JMP -8

Exam Tip: ARM Has No Absolute Addressing

The ARM instruction set does *not* support absolute (direct) addressing, where the operand address is embedded as a literal in the instruction. All memory accesses on ARM use a register to hold the base address (register indirect and its variants). This distinguishes ARM from Intel architectures.

3.1.4 Role and Categories of Instructions

The role of an instruction is to make **purposeful changes** to the **current state** of the processor. The visible state of a processor is defined by its *programmer's model* together with the contents of memory. There are three broad categories of instructions available in a typical processor:

Data Processing

Arithmetic and logical operations that compute new values from existing register contents. ARM examples: ADD R0, R1, R2; SUB R1, R2, #3; EOR R3, R3, R2.

Data Transfer

Move data between registers, or between registers and memory. ARM examples: MOV R1, R0; STR R0, [R2, #4]; LDR R1, [R2].

Program Control

Alter the flow of execution by modifying the Program Counter. ARM examples: B Back (unconditional branch); BNE Loop (branch if not equal); BL Routine (branch with link, i.e. subroutine call).

3.1.5 The ARM MOV Instruction

Definition 3.1.2 (MOV Instruction). The ARM **MOV** instruction is a two-operand, register-to-register data transfer. It copies the value of the *source* operand (right) into the *destination* register (left):

$$\text{MOV } R_d, R_s \implies R_d \leftarrow R_s$$

The source register is unchanged. For example, after MOV R1, R0, both R0 and R1 hold the value that was originally in R0.

Warning: ARM Operand Order

In ARM assembly, the **destination is always the leftmost operand** and the **source is the rightmost**. This is the opposite of the Intel convention for some instructions, and mixing up the order is a common mistake.

3.1.6 The ARM Programmer’s Model

Definition 3.1.3 (Programmer’s Model). The **programmer’s model** is the set of registers visible to software. It defines the *state* that instructions can read and modify. In the ARM architecture, the programmer’s model in User mode consists of 16 general-purpose 32-bit registers (R0–R15) plus the Current Processor Status Register (**CPSR**).

The registers have the following roles:

- R0–R12 — general-purpose registers, freely usable by the programmer for data and addresses.
- R13 (**Stack Pointer, SP**) — by convention, holds the address of the current top of the hardware stack.
- R14 (**Link Register, LR**) — holds the return address when a subroutine is called via a Branch-with-Link (BL) instruction.
- R15 (**Program Counter, PC**) — holds the address of the next instruction to be fetched.
- **CPSR (Current Processor Status Register)** — holds the condition code flags **N**, **Z**, **V**, **C** and various processor control bits.

3.1.7 Program Counter (PC / R15)

R15 is the Program Counter. Its value at any moment is the address of the **next instruction to be fetched** from code memory. The PC is updated automatically after each instruction fetch: for the ARM’s 32-bit (4-byte) instructions it increments by 4, so that instructions are executed sequentially by default.

Sequential execution can be broken by a **branch** or **jump** instruction, which loads a new address into the PC, transferring control to a non-sequential location.

Remark 3.1.4 (ARM Pipeline Offset). Due to the ARM’s three-stage pipeline (Fetch — Decode — Execute), the value read from the PC during execution of an instruction is the address of that instruction *plus 8 bytes* (i.e. two instructions ahead). This is a known peculiarity of the ARM that must be accounted for when computing PC-relative addresses manually.

3.1.8 Stack Pointer (SP / R13) and Link Register (LR / R14)

Stack Pointer. By convention, R13 is designated the **Stack Pointer (SP)**. The SP holds the address of the top of the **stack** — a region of main memory used to temporarily save register values, pass function arguments, and store local variables. Typical stack operations *push* data to the stack (decrement SP, then write) or *pop* data from it (read, then increment SP).

Link Register. R14 is the **Link Register (LR)**. When a BL (Branch with Link) instruction is executed to call a subroutine, the processor automatically copies the current PC (R15) into R14. On return from the subroutine, the saved address is moved back into the PC (typically via `MOV PC, LR`), resuming execution after the call site. At all other times R14 may be used as a general-purpose register.

3.1.9 CPSR Condition Code Flags

The four condition code flags occupy the most-significant bits of the CPSR:

Bit	Flag	Set when ...
31	N (Negative)	The result of the last flag-setting operation was negative (MSB of result = 1).
30	Z (Zero)	The result of the last flag-setting operation was zero.
29	C (Carry)	The last flag-setting operation produced a carry-out of the most significant bit (unsigned overflow).
28	V (Overflow)	The last flag-setting operation produced a signed arithmetic overflow.

The flags are only updated when the instruction mnemonic includes an **S** suffix (e.g. **ADDs**, **SUBs**) or for comparison instructions (**CMP**, **TST**). They are used by conditional branch instructions (e.g. **BNE** checks $Z = 0$, **BMI** checks $N = 1$) to implement loops and **if**-statements.

Key Result: NZVC Flags

N = Negative, Z = Zero, C = Carry (unsigned overflow), V = oVerflow (signed overflow). These four bits in the CPSR are set by arithmetic/logic instructions with the **S** suffix and tested by all conditional branch instructions.

3.2 Basic Execution Cycle

3.2.1 The LDR Instruction

Definition 3.2.1 (LDR Instruction). The ARM **LDR** (Load Register) instruction copies a 32-bit word from *memory* into a register. The left operand is always the destination register; the right operand (in square brackets) is the source memory address, held indirectly in a register (register-indirect addressing):

$$\text{LDR } R_d, [R_a] \implies R_d \leftarrow \text{Mem}[R_a]$$

For example, if $R_0 = 0x00000100$ and memory at address $0x100$ contains the four bytes $0xDD$, $0xCC$, $0xBB$, $0xAA$ (little-endian), then after **LDR R1, [R0]**, register R1 holds $0xAABBCCDD$.

Exam Tip: LDR vs. MOV

MOV copies data *between registers*; **LDR** copies data from *memory into a register*. On ARM, you cannot directly read a memory location into a register using **MOV** — **LDR** (or a variant with an offset) is required.

3.2.2 The Role of the Processor and the Fetch–Decode–Execute Cycle

The fundamental job of the CPU is to repeatedly execute the **fetch–decode–execute (FDE)** cycle:

1. **Fetch instruction** — the CPU places the address in the PC onto the address bus and reads the instruction word from code memory into the **Instruction Register (IR)**. The PC is then incremented by 4.
2. **Decode instruction** — the **Control Unit (CU)** decodes the opcode in the IR to determine what action is required.

3. **Fetch data** (optional) — if the decoded instruction requires a memory operand (e.g. an LDR), the CPU places the data address on the address bus and reads the operand into a **Buffer Register (BF)**.
4. **Execute** — the CPU performs the required operation (arithmetic, logic, or data move), writing the result to the destination register or memory location.

This cycle repeats continuously from power-on until the CPU is halted or reset.

3.2.3 A Simple Processor: Internal Structure

The basic components of a simple CPU and its interface to main memory are:

- **General-purpose registers** — fast on-chip storage (e.g. R0–R12 on ARM).
- **Arithmetic Logic Unit (ALU)** — performs arithmetic and logical operations.
- **Buffer Register (BF)** — a staging register used to hold operands fetched from memory before they are transferred to a destination register.
- **Instruction Register (IR)** — holds the currently-executing instruction word after it is fetched from memory.
- **Data Address Register (DAR)** — holds the effective memory address of a data operand during the operand-fetch phase.
- **Program Counter (PC)** — holds the address of the next instruction; drives the address bus during the instruction-fetch phase.
- **Control Unit (CU)** — decodes the opcode in the IR and generates the internal control signals and external bus control signals (Read/Write) needed to execute the instruction.
- **Address Buffer (AB) / Data Buffer (DB)** — interface latches that connect the internal address and data buses to the external bus driving main memory.

3.2.4 Execution of LDR R1, [R0]: Step-by-Step

Consider the instruction LDR R1, [R0] stored at code address 0x000 (machine code 0xE5901000), with R0 = 0x00000100 and the data value 0xAABBCCDD stored at data address 0x100.

Phase 1 — Instruction Fetch.

1. The PC (0x000) is placed on the internal address bus and driven onto the external address bus via AB.
2. The CU asserts a *Read* signal on the control bus.
3. The memory responds by placing the four bytes of the instruction on the data bus; they are clocked into the IR via DB. The CU decodes the opcode.

Phase 2 — Operand Fetch.

4. Decoding reveals that LDR R1, [R0] requires a memory operand. The content of R0 (0x00000100) is transferred to the DAR and placed on the address bus.
5. The CU asserts another *Read* signal.
6. The data value 0xAABBCCDD is read from memory and loaded into the Buffer Register BF.

Phase 3 — Execute (Load).

7. The CU generates an internal control signal to transfer the contents of BF into the destination register R1.
8. After execution, $R1 = 0xAABBCCDD$.

3.2.5 Performance Considerations

Execution of a single instruction may involve **multiple memory accesses**. In a von Neumann (single-memory) CPU, different instructions take different numbers of clock cycles because some require an extra operand-fetch phase.

The **von Neumann bottleneck** (introduced in Definition 1.3.2) directly limits performance here: data transfers on the external bus are significantly slower than operations on the CPU's internal bus. System throughput is therefore bounded by the memory access bandwidth, not by the raw speed of the ALU.

Two architectural strategies address this limitation:

- **Keeping operands in registers** — data already in R0–R12 can be used immediately without a memory access, eliminating the operand-fetch phase. This is why *register allocation* is a key optimisation in compilers.
- **Harvard architecture** (see Definition 1.3.1) — separating code and data into two independent memories allows the CPU to fetch the next instruction and the current data operand in *parallel*, making the instruction execution time more uniform and increasing throughput.

Key Result: Execution Speed Factors

The number of clock cycles an instruction takes depends primarily on how many **memory accesses** are required. Instructions that operate only on registers (e.g. `MOV R1,R0`) are faster than instructions that require a memory operand (e.g. `LDR R1,[R0]`), because the latter needs an additional bus cycle. The memory-access bottleneck is a direct consequence of the von Neumann architecture.

Chapter 4

Addressing Modes

4.1 Introduction to Assembly Programming

Unlike high-level programming languages, assembly language statements operate at the level of the processor’s instruction set architecture (ISA) directly. Such statements are known as **mnemonics**: each mnemonic has a one-to-one correspondence with a binary pattern (*machine code*) that is directly understood by the CPU. Because mnemonics are **hardware-dependent**, they reference CPU registers by name and are converted to machine code by a program called an **assembler**.

4.1.1 Why Use Assembly Language?

Although high-level languages such as C are sufficient for most tasks, there are situations where assembly-level coding provides decisive advantages:

- **Faster execution speed** — Algorithms for real-time signal processing in handheld devices can be computationally demanding, and hand-crafted assembly may exploit processor features that a compiler misses.
- **More compact program size** — Low-cost embedded devices may have small memory capacity but require many functionalities.
- **Exploitation of optimised ISA features** — High-level language compiled code may not exploit optimised instructions, addressing modes, and features available in the processor ISA to produce efficient run-time code.
- **Cybersecurity** — Many cybersecurity roles require good knowledge of assembly programming for reverse engineering, vulnerability analysis, and exploit development.

4.1.2 When to Use Assembly Language?

Assembly is particularly beneficial for:

- **Critical parts of the operating system** — Especially parts of the system kernel that are constantly executed, such as the scheduler and interrupt handlers.
- **Input/Output intensive code** — Device drivers and “loopy” segments of code that process streaming data (e.g. video decoders).
- **Time-critical code** — Code that detects incoming sensor signals and responds rapidly, e.g. the Anti-lock Brake System (ABS) in cars.

Example 4.1.1 (C to ARM Assembly). The following C fragment and its ARM assembly equivalent illustrate the one-to-one nature of assembly mnemonics:

```

1 if (a > c)
2     b = a;
3 else
4     b = c;

```

```

1     CMP    R0, R2
2     BLE    Else
3     MOV    R1, R0    ; b = a
4     B      Skip
5 Else     MOV    R1, R2    ; b = c
6 Skip:

```

4.2 Addressing Modes — Overview

Definition 4.2.1 (Addressing Mode). An **addressing mode** (AM) is concerned with *how data is accessed*, not the way data is processed. The correct AM allows the CPU to identify the actual operand, or the address location where the operand is stored.

The ARM processor ISA supports the following addressing modes:

1. Register direct
2. Immediate data
3. Register indirect
4. Register indirect with offset
5. Register indirect with index register
6. Pre- and post-autoindexing

The table below gives a side-by-side comparison of ARM and Intel addressing mode syntax.

Addressing Mode	ARM example	Intel example
Absolute (Direct)	(none)	MOV AX, [1000h]
Register Direct	MOV R1, R0	MOV AX, DX
Immediate	MOV R1, #3	MOV AX, 0003h
Register Indirect	LDR R1, [R0]	MOV AX, [BX]
Register Indirect + Offset	LDR R1, [R0, #4]	MOV AX, [BX+4]
Register Indirect + Index	LDR R1, [R0, R2]	MOV AH, [BX+DI]
Implied	BNE LOOP	JMP -8

4.3 Register Direct Addressing

In register direct addressing, the operand *is the content of the specified register*. Register direct may be used for both the destination and the source operand.

```

1 MOV R1, R0    ; R1 <- R0

```

In the MOV instruction the *right* operand is the source and the *left* operand is the destination. This is a **fast** addressing mode since no further memory access is involved during execution — it should therefore be preferred whenever possible to optimise execution speed.

All 16 ARM registers (R0–R12, SP/R13, LR/R14, PC/R15) may serve as either source or destination:

```

1 MOV PC, R1      ; jump to address stored in R1
2 MOV R3, LR      ; save link register in R3
3 MOVS R0, R0     ; test N or Z condition of R0

```

Key Result: Register Direct is Fastest

Register direct involves **no memory access** during execution, making it the fastest addressing mode. Use it to hold frequently accessed variables in registers rather than memory.

4.4 Immediate Addressing

In immediate addressing, the operand is **directly specified within the instruction itself**. The “#” symbol precedes the immediate value. Immediate addressing can only be used as a *source* operand.

```

1 MOV R1, #3      ; R1 <- 3

```

Immediate addressing is used to load **constant values** known at the time of coding (e.g. loading a loop counter).

4.4.1 Encoding of the 12-bit Immediate Field

In a 32-bit ARM instruction, only 12 bits are available for the immediate operand. These 12 bits encode a value as an **8-bit base** (range 0–255) *rotated right* by $2n$ bits, where n is a 4-bit field ($0 \leq n \leq 15$). Consequently only a *subset* of all 2^{32} possible values can be represented directly.

- Any value whose bits fit within 8 bits (i.e. 0–255) is always valid: `MOV R3, #0xFF`.
- Values expressible as an 8-bit quantity rotated right by an even number of bits are valid: e.g. `0x100 = 1` rotated right by 24 ($n = 12$) is valid.
- Values that cannot be expressed in this form are **invalid** as a single immediate: `0x102` is invalid because `0x102 = 0b1_0000_0010` requires two non-contiguous bit groups.

Warning: Not All Constants Are Valid Immediates

The assembler will warn if a requested immediate value cannot be encoded. A combination of instructions must then be used:

```

1 MOV R1, #0x100 ; load 0x100 into R1
2 ADD R1, R1, #2 ; add 2 to get 0x102

```

Key Result: Immediate Addressing Efficiency

Like register direct, immediate addressing in the ARM incurs **no extra memory access** during execution (the operand is fetched as part of the instruction fetch). However, because the data is encoded within a fixed-length 32-bit instruction, only a *subset* of all possible 32-bit values are available as immediate operands.

4.5 Register Indirect Addressing

4.5.1 Motivation

Neither register direct nor immediate addressing allows the CPU to access operands *stored in memory*. Because a 32-bit ARM instruction cannot itself contain a full 32-bit memory address alongside an opcode, the ARM ISA instead stores the 32-bit address in a register. This register then *points to* the memory location where the operand resides — the standard register indirect pattern. The ARM uses the LDR and STR mnemonics for memory access.

4.5.2 The LDR Instruction

Definition 4.5.1 (LDR — Load Register). LDR Rd, [Rn] copies a 32-bit word from the memory address contained in register Rn into the destination register Rd.

```
1 LDR R1, [R0] ; R1 <- mem[R0]
```

4.5.3 The STR Instruction

Definition 4.5.2 (STR — Store Register). STR Rs, [Rn] copies the 32-bit content of source register Rs to the memory address contained in Rn.

```
1 STR R1, [R0] ; mem[R0] <- R1
```

4.5.4 Data Alignment Constraint

A 32-bit memory access must follow the **data alignment constraint**: the four-byte word must start at an address that is a multiple of 4. An unaligned access results in undefined behaviour or a performance penalty depending on the ARM variant.

Warning: Unaligned Memory Access

Always ensure that the base address in the indirect register is a multiple of 4 when performing 32-bit LDR/STR operations. Unaligned accesses invariably result in **performance degradation** (and may cause a hardware fault on some ARM cores).

4.5.5 Register Indirect with Offset (Base + Offset)

A fixed offset can be added to the base register to compute the **effective address (EA)** without modifying the base register itself:

$$EA = R_{\text{base}} + \text{offset}$$

```
1 LDR R1, [R0, #4] ; R1 <- mem[R0 + 4], R0 unchanged
```

This is useful when the array element's index is **known at coding time**.

Example 4.5.3 (Accessing First and Last Array Elements). The following ARM code assigns the value 7 to i[0] and i[4] of an integer array whose base address is 0x100:

```
1 MOV R2, #0x100 ; base address of array i
2 MOV R1, #7      ; value to store
3 STR R1, [R2, #0] ; i[0] = 7
4 STR R1, [R2, #16] ; i[4] = 7 (offset = 4*4 = 16 bytes)
```

Note that each `int` occupies 4 bytes, so element k has byte offset $4k$.

4.5.6 Register Indirect with Index Register (Base + Index)

When the array element's position is computed *at run time*, the index is held in a register and added to the base register:

$$EA = R_{\text{base}} + R_{\text{index}}$$

```
1 LDR R1, [R0, R2] ; R1 <- mem[R0 + R2], R0 unchanged
```

Example 4.5.4 (Clearing All Elements of a 400-element Array).

```
1 MOV R2, #0x100 ; base address of i
2 MOV R0, #0      ; value 0 to store
3 MOV R1, #0      ; index register (byte offset)
4 loop STR R0, [R2, R1] ; i[n] = 0
5 ADD R1, R1, #4 ; advance index by 4 bytes
6 ; (loop back 399 times)
```

This loop executes 3 instructions $\times$ 400 iterations = 1200 cycles.

Key Result: Base + Offset vs. Base + Index

- Use **base + offset** when the array element position is known at coding time (fixed offset compiled into the instruction).
- Use **base + index register** when the element position is computed during execution (variable index stored in a register).
- In both cases, the **base register is not modified** after execution.

4.6 Register Indirect with Autoindexing

4.6.1 Concept

In the addressing modes covered so far, the base register is never modified. **Autoindexing** extends register indirect by allowing the indirect register's content to be *updated during execution*, providing an efficient way to traverse consecutive array elements without a separate `ADD` instruction.

The “!” suffix on an ARM instruction signals that the autoindex register should be updated with the computed effective address.

4.6.2 Pre-index vs. Post-index

Definition 4.6.1 (Pre-index). In **pre-index** mode the indirect register is updated *before* being used to compute the effective address:

$$R_0 \leftarrow R_0 + \text{offset}, \quad \text{then} \quad R_1 \leftarrow \text{mem}[R_0]$$

ARM syntax: `LDR R1, [R0, #4]!`

Definition 4.6.2 (Post-index). In **post-index** mode the current register value is used to compute the effective address *first*, then the register is updated:

$$R_1 \leftarrow \text{mem}[R_0], \quad \text{then} \quad R_0 \leftarrow R_0 + \text{offset}$$

ARM syntax: `LDR R1, [R0], #4`

The four autoindexing variants are summarised below:

Variant	ARM syntax	Semantics
Offset, pre-index	LDR R1, [R0, #4] !	R0=R0+4; R1=mem[R0]
Index reg, pre-index	LDR R1, [R0, R2] !	R0=R0+R2; R1=mem[R0]
Offset, post-index	LDR R1, [R0], #4	R1=mem[R0]; R0=R0+4
Index reg, post-index	LDR R1, [R0], R2	R1=mem[R0]; R0=R0+R2

Example 4.6.3 (Optimised Array Clear with Pre-index Autoindexing). Using pre-index autoindexing reduces the loop to 2 instructions:

```

1      MOV R2, #0x100    ; base address
2      MOV R1, #0        ; value to store
3      STR R1, [R2]      ; i[0] = 0
4 loop STR R1, [R2, #4]! ; R2 += 4; i[n] = 0
5      ; (loop back 398 times)

```

Total: $2 \times 400 = 800$ cycles (vs. 1200 using base + index).

Example 4.6.4 (Alternative Clear with Post-index Autoindexing). Post-index keeps *all* array stores inside the loop:

```

1      MOV R2, #0x100    ; base address
2      MOV R1, #0        ; value to store
3 loop STR R1, [R2], #4 ; i[n] = 0; R2 += 4
4      ; (loop back 399 times)

```

Total: $2 \times 400 = 800$ cycles.

Exam Tip: Pre- vs. Post-index Side Effects

In pre-index (!), the base register is updated *before* the memory access, so the first iteration accesses **base + offset**. In post-index, the *current* base is accessed first; the update happens after. Confusing these leads to an off-by-one error in the first or last array element.

4.7 The System Stack and Stack Implementations

Definition 4.7.1 (Stack). A **stack** is a first-in, last-out (FILO) linear data structure maintained in the memory's data area. The ARM system stack is managed by the dedicated **stack pointer** SP (alias R13).

The ARM ISA supports four stack implementations based on two orthogonal properties:

- **Full (F) vs. Empty (E)**: “Full” means SP points to the *last occupied* slot; “Empty” means SP points to the *next free* slot.
- **Descending (D) vs. Ascending (A)**: The stack grows towards lower memory addresses (descending) or higher memory addresses (ascending).

This gives four combinations: **FD, FA, ED, EA**.

4.7.1 Full Descending (FD) Stack — Default ARM Stack

The FD stack is the default ARM convention. SP starts at a high address (e.g. 0xFF000000 in the VisUAL simulator) and grows *downward*. SP always points to the top-most valid (full) item. Push/pop code for the FD stack is covered in detail in Section 6.5.1.

4.7.2 Empty Ascending (EA) Stack — Alternative Implementation

In the EA stack, SP points to the *next free* slot and the stack grows toward higher addresses. Push uses post-index autoindexing (store then increment); pop uses pre-index autoindexing (decrement then load) — the opposite sequence to FD.

Warning: Push and Pop Must Be Complementary

For a given stack implementation, the **Push** and **Pop** operations must always be complementary so that the stack grows and collapses correctly. Mixing conventions (e.g. using an FD push with an EA pop) will corrupt the stack pointer.

4.8 PC-related Addressing Modes

4.8.1 Absolute vs. Relative Jumps

The program counter PC (R15) holds the address of the next instruction to be fetched. Non-sequential execution is achieved by modifying PC.

Absolute jump. Loading a fixed address directly into the PC causes an *absolute jump*:

```
1 MOV PC, #0x060 ; jump unconditionally to address 0x060
```

Absolute jumps are **not position-independent**: the code can only execute correctly if loaded at a specific memory location.

Relative jump. The B (Branch) instruction adds a *signed offset* to the current PC value:

```
1 B CodeB ; PC <- PC + signed_offset
```

The offset range in ARM is ± 32 MiB. Because of the ARM pipeline, PC points *8 bytes ahead* of the currently executing instruction when the branch is evaluated — this must be accounted for when computing offsets manually.

Remark 4.8.1 (ARM Pipeline PC Offset). Recall from Section 3.1.4 that in the ARM pipeline PC always reads as the address of the *current instruction plus 8*. This applies equally to branch offset calculations and to PC-relative data addressing.

4.8.2 Position-Independent Code

Definition 4.8.2 (Position-Independent Code). **Position-independent (P-I) code** can be loaded anywhere in memory and execute correctly, because all addresses are expressed as *offsets relative to the current PC* rather than as absolute values.

Relative jumps (using B) produce position-independent code because the offset between the branch instruction and its target is preserved regardless of where the code is loaded.

4.8.3 PC-Relative Data Addressing

To access variables in the data segment in a position-independent manner, the ADD instruction is used to compute the variable's address relative to the PC:

```
1 ADD R0, PC, #0x0F8 ; R0 <- PC + 8 + 0x0F8 = address of Var1
2 LDR R1, [R0] ; R1 <- mem[Var1]
```

The PC-relative offset is calculated as:

$$\text{offset} = \text{Var address} - (\text{current PC value} + 8)$$

Exam Tip: PC-Relative Offset Calculation

Always add 8 to the current instruction's address when computing a PC-relative offset, because of the ARM pipeline's fetch-ahead. Forgetting the +8 is the most common mistake when hand-assembling PC-relative load/store instructions.

Key Result: Position-Independent Code Summary

- Use **relative branch** (B/BL) instead of loading a fixed address into PC for jumps.
- Use **PC-relative addressing** (ADD Rd,PC,#offset followed by LDR/STR) instead of absolute addresses to access data.
- Both techniques produce code that remains correct when relocated to a different memory region.

Chapter 5

Instruction Set

Non-system-level instructions in a processor fall into three categories:

- **Data transfer** — instructions that move data between registers and/or memory.
- **Data processing** — instructions that modify register contents through arithmetic, logical, or shift operations.
- **Program control** — instructions that alter the normal sequential execution flow by modifying the Program Counter (PC).

Chapters 5–onwards treat each category in turn, focusing on the ARM instruction set.

5.1 Data Transfer Instructions

Data transfer instructions move data between registers or between a register and memory.

5.1.1 Register Data Transfer: MOV and MVN

The MOV instruction copies a source operand into a destination register. The source may use *register direct* or *immediate* addressing:

```
1 MOV R1, R0      ; copy R0 into R1 (register direct)
2 MOVS R0, #0     ; move 0 into R0 and set Z flag
3 MVN R1, R0      ; R1 = NOT(R0) (bitwise complement)
4 MVN R0, #0     ; R0 = 0xFFFFFFFF (i.e. -1 in 2's complement)
```

Appending the **S** suffix causes the instruction to update the **N** and **Z** condition code flags. MVN (Move Negative / complement) performs a bitwise NOT on the source before storing the result.

Key Result: MOV vs. MVN

MOV Rd, src copies src unmodified into Rd.

MVN Rd, src stores ~src (bitwise complement) into Rd.

Both support register-direct and immediate addressing; only the rightmost operand may be an immediate.

5.1.2 Memory Data Transfer: LDR and STR

Data in memory is transferred via LDR (load) and STR (store), which use various *register indirect* addressing modes (see Section 4.5). Memory data transfer instructions require **two clock cycles** to execute.

```
1 LDR R1, [R0]     ; load 32-bit word at address in R0 into R1
2 LDRB R2, [R0]    ; load byte at address in R0 into R2 (zero-extended)
3 STR R1, [R0]     ; store 4 bytes from R1 to address in R0
4 STRB R2, [R0, #1]! ; store byte in R2 to [R0+1], then R0 = R0+1
```

Key points about byte access:

- A byte loaded with LDRB is **zero-extended** to fill the 32-bit register.
- Byte access (LDRB/STRB) has **no data alignment restrictions**.
- Word access (LDR/STR) is subject to 4-byte alignment (see Section 2.6).

Exam Tip: Byte vs. Word Transfer

Use LDRB/STRB when operating on individual bytes (e.g. ASCII characters). The loaded byte is always zero-extended — there is no sign-extension variant on ARMv4.

5.1.3 Program Example: Copying a Block of Memory

Block copy replicates a contiguous memory segment from one location to another using MOV, LDR, and STR.

Basic version (30 cycles for 5 words):

```
1 MOV R0, #0x100    ; source pointer
2 MOV R1, #0x200    ; destination pointer
3 ; loop (repeat 5 times):
4 LDR R2, [R0]       ; memory -> register
5 STR R2, [R1]       ; register -> memory
6 ADD R0, R0, #4     ; advance source pointer
7 ADD R1, R1, #4     ; advance destination pointer
```

Optimised with post-index autoindexing (20 cycles for 5 words): Using the post-index autoindexing mode (Section 4.6.2) eliminates the explicit ADD pointer increments:

```
1 MOV R0, #0x100
2 MOV R1, #0x200
3 ; loop (repeat 5 times):
4 LDR R2, [R0], #4   ; load word, then R0 = R0 + 4
5 STR R2, [R1], #4   ; store word, then R1 = R1 + 4
```

Further optimisation — single pointer register (20 cycles, one register fewer): Using base-plus-offset addressing, only one pointer register is needed. The offset is the fixed distance between source and destination arrays: $0x200 - 0x100 + 4 = 0xFC$.

```
1 MOV R0, #0x100
2 ; loop (repeat 5 times):
3 LDR R2, [R0], #4   ; load from source, auto-increment R0
4 STR R2, [R0, #0xFC] ; store to destination via offset
```

Warning: Offset Range Limit

The immediate offset for register indirect addressing is limited to ± 4096 bytes. When source and destination arrays are more than 4096 bytes apart, this single-pointer technique cannot be used.

5.2 Arithmetic Instructions

Arithmetic instructions are **three-operand** instructions that modify register contents through addition or subtraction. The rightmost operand (second source) may be a register or an immediate value; all other operands must be registers.

The ARM arithmetic instruction set includes:

- ADD — addition
- SUB — subtraction
- RSB — reverse subtraction (swaps operand order)
- ADC — add with carry
- SBC — subtract with carry
- RSC — reverse subtract with carry

Appending the **S suffix** to any arithmetic instruction causes it to update all four condition code flags: N, Z, V, C.

5.2.1 ADD Instruction

Addition is a *commutative* operation — operand order does not affect the result. Only the *rightmost* operand may take an immediate value.

```
1 ADD R2, R0, R1 ; R2 = R0 + R1
2 ADD R2, R0, #4 ; R2 = R0 + 4
3 ADDS R2, R0, #8 ; R2 = R0 + 8; update N,Z,V,C
```

Condition code flags and ADD. ADD can affect all four flags:

- **V flag** (overflow): set when two operands with the *same* sign produce a result with the *opposite* sign — indicating a signed overflow.

$$\underbrace{0000\ 0001}_{+1} + \underbrace{0111\ 1111}_{+127} = \underbrace{1000\ 0000}_{-128} \Rightarrow N=1, V=1$$

- **C flag** (carry): set when an unsigned overflow occurs, i.e. a carry out of bit 31.

$$\underbrace{0000\ 0001}_1 + \underbrace{1111\ 1111}_{255} = \underbrace{0000\ 0000}_0 + \text{carry} \Rightarrow Z=1, C=1$$

Key Result: V flag vs. C flag for ADD

V = 1 $\Rightarrow$ signed overflow (result wrong for signed arithmetic).

C = 1 $\Rightarrow$ unsigned overflow (result wrong for unsigned arithmetic, i.e. carry out of the 32-bit word).

5.2.2 SUB and RSB Instructions

SUB is *non-commutative*: SUB R2,R0,R1 computes $R2 = R0 - R1$. Internally, subtraction is performed as addition of the two's complement:

$$A - B = A + (-B) = A + \overline{B} + 1$$

```
1 SUB  R2, R0, R1    ; R2 = R0 - R1
2 SUBS R2, R0, #4    ; R2 = R0 - 4; update N,Z,V,C
3 RSB  R2, R0, #4    ; R2 = 4 - R0 (immediate as minuend)
4 RSBS R2, R0, #0    ; R2 = 0 - R0 (negate R0); update flags
```

RSB (Reverse Subtract) is useful when an immediate value must serve as the *minuend* — something SUB cannot achieve directly.

Condition code flags and SUB.

- **C flag (borrow indicator)**: In ARM's SUB, $C = 0$ indicates a borrow (unsigned underflow: $\text{unsigned}(A) < \text{unsigned}(B)$); $C = 1$ means no borrow.
- **V flag**: Set ($V = 1$) when the signed result is outside $[-2^{31}, 2^{31})$.

Warning: C flag polarity differs for SUB vs. ADD

For ADD, $C = 1$ means unsigned overflow. For SUB, $C = 0$ means unsigned underflow (borrow) — the polarity is *inverted*. Always consult the ARM Architecture Reference Manual for exact flag semantics.

Program Example: Convert Negative to Positive. The following code negates 10 signed integers stored at 0x100. The optimised version replaces MVN + ADD with a single RSB:

```
1 MOV  R0, #0x100      ; source pointer
2 ; loop (repeat 10 times):
3 LDR  R1, [R0]         ; load next negative integer
4 RSB  R1, R1, #0       ; R1 = 0 - R1 (negate)
5 STR  R1, [R0], #4     ; store result, advance pointer
```

5.2.3 Carry-Based Arithmetic: ADC, SBC, RSC

The carry-based variants extend ADD, SUB, and RSB to support **multi-precision arithmetic** — computations on values wider than 32 bits by chaining multiple instructions through the carry flag:

```
1 ADC  R2, R0, R1      ; R2 = R0 + R1 + C    (ADD with carry)
2 SBC  R2, R0, R1      ; R2 = R0 - R1 + NOT(C) (SUB with carry)
3 RSC  R2, R0, R1      ; R2 = R1 - R0 + NOT(C) (RSB with carry)
```

Like all arithmetic instructions, the S suffix can be appended to update the N, Z, V, C flags.

5.3 Logical, Shift, and Rotate Instructions

5.3.1 Logical Instructions: AND, ORR, EOR, MVN

ARM provides Boolean operations as three-operand instructions (except MVN, which is two-operand):

```

1 MVNS R2, R2          ; R2 = NOT(R2); set N, Z flags
2 ORRS R1, R1, #0x0000FFFF ; R1 |= 0x0000FFFF; set N, Z
3 AND  R1, R0, #0xF0    ; clear bits 0-3 of R0, store in R1
4 ORR  R0, R0, #0xF0    ; set bits 4-7 of R0
5 EOR  R2, R0, #0xF0    ; complement bits 4-7 of R0, store in R2

```

The three principal uses are:

- **AND** with a **0 mask** — clears specific bits (mask bit = 0 $\Rightarrow$ output = 0).
- **ORR** with a **1 mask** — sets specific bits (mask bit = 1 $\Rightarrow$ output = 1).
- **EOR** with a **1 mask** — complements (toggles) specific bits.

The S suffix on logical instructions influences only the N and Z flags.

Exam Tip: Bit-Manipulation Masks

To **clear** bit n : AND Rd, Rs, #~(1 << n) (all ones except bit n).

To **set** bit n : ORR Rd, Rs, #(1 << n).

To **toggle** bit n : EOR Rd, Rs, #(1 << n).

Program Example: Alternate LED Flashing. Green and Red LEDs are mapped to bits 3 and 2 of R0 (active low). Once the initial pattern is established, EOR can toggle both bits every cycle:

```

1 AND R0, R0, #0xFFFFF0B ; turn on Red (clear bit 2)
2 ORR R0, R0, #0x00000008 ; turn off Green (set bit 3)
3 ; output pattern and delay
4 Loop:
5 EOR R0, R0, #0x0000000C ; flip bits 3 and 2 simultaneously
6 ; output pattern and delay
7 ; loop back

```

5.3.2 Shift and Rotate Operations

ARM provides five shift/rotate modes, all applied to the *rightmost* operand of a data transfer or processing instruction via the **Barrel Shifter**:

- **LSL** — Logical Shift Left: zeros enter from the right; MSB exits to C.
- **LSR** — Logical Shift Right: zeros enter from the left; LSB exits to C.
- **ASR** — Arithmetic Shift Right: *sign bit* replicates from the left; LSB exits to C. Used for signed division.
- **ROR** — Rotate Right: the bit shifted out at LSB re-enters at MSB and also updates C.
- **RRX** — Rotate Right Extended (1-bit only): C enters at MSB; LSB exits to C.

Shift amount is specified as an *immediate* or a *register* (dynamic shift):

```

1 MOV R0, R0, LSL #1 ; R0 = R0 << 1 (multiply by 2)
2 ADD R2, R1, R0, LSR #2 ; R2 = R1 + (R0 >> 2)
3 ADDS R2, R1, R0, LSL R4 ; R2 = R1 + (R0 << R4); update flags
4 ORRS R0, R0, R0, ROR #1 ; R0 |= (R0 ROR 1); update C

```

Arithmetic with shifts. A left shift by N bits multiplies by 2^N ; a right shift by N bits divides by 2^N :

$$00000100_2 (4) \xrightarrow{\text{LSL } \#2} 00010000_2 (16) \qquad 00000100_2 (4) \xrightarrow{\text{LSR } \#1} 00000010_2 (2)$$

Use ASR for signed division to preserve the sign bit.

Key Result: Shift as Fast Multiplication/Division

LSL #N $\equiv$ multiply by 2^N (signed and unsigned).
 LSR #N $\equiv$ divide by 2^N (unsigned only — zeros fill from left).
 ASR #N $\equiv$ divide by 2^N (signed — sign bit fills from left).

Program Example: Count Number of 1s (Optimised). LSR shifts each bit of R0 into the carry flag one at a time; ADC accumulates the carry into a counter. The loop terminates naturally when R0 becomes zero (Z flag set by MOVs):

```

1 MOV R2, #0          ; clear 1-counter
2 Loop:
3 MOVS R0, R0, LSR #1 ; right-shift R0 one bit into C; set Z if R0=0
4 ADC R2, R2, #0       ; R2 = R2 + 0 + C (add carry to counter)
5 BNE Loop            ; loop while R0 != 0

```

5.4 Program Control Instructions

Program control instructions alter the normal sequential execution flow by modifying the Program Counter (PC) directly or through a branch instruction.

5.4.1 Conditional Branch: Bcc

ARM's Bcc (Branch if condition cc) adds a signed displacement to the PC when the specified condition is true; otherwise execution continues at the next instruction.

```

1 BEQ Skip          ; branch to Skip if Z = 1 (result was zero / equal)
2 BNE Loop          ; branch to Loop if Z = 0 (result was non-zero)
3 BGT Else          ; branch to Else if result > 0 (signed)
4 B   Back          ; unconditional branch (always)

```

Key implementation details:

- Bcc uses **PC-relative addressing** with a displacement range of ± 32 MB.
- The PC value used to compute the displacement is **8 bytes ahead** of the current Bcc instruction (ARM pipeline effect — recall Section 4.8.1).
- Branch targets are specified as **address labels**; the assembler computes the displacement.

Complete list of condition suffixes.

Suffix	Flag condition	Meaning
EQ	$Z = 1$	Equal (zero)
NE	$Z = 0$	Not equal
CS/HS	$C = 1$	Higher or same (unsigned $\geq$)
CC/LO	$C = 0$	Lower (unsigned $<$)
MI	$N = 1$	Negative
PL	$N = 0$	Positive or zero
VS	$V = 1$	Overflow
VC	$V = 0$	No overflow
HI	$C = 1$ and $Z = 0$	Higher (unsigned $>$)
LS	$C = 0$ or $Z = 1$	Lower or same (unsigned $\leq$)
GE	$N = V$	Greater than or equal (signed $\geq$)
LT	$N \neq V$	Less than (signed $<$)
GT	$Z = 0$ and $N = V$	Greater than (signed $>$)
LE	$Z = 1$ or $N \neq V$	Less than or equal (signed $\leq$)
AL	(any)	Always (default)

Exam Tip: Signed vs. Unsigned Branches

For **signed** comparisons use GT, LT, GE, LE.
For **unsigned** comparisons use HI, LO, HS, LS.
Mixing them up (e.g. using BGT on unsigned values) gives incorrect results when the MSB of one operand is set.

5.4.2 Conditional Test Instructions: CMP, CMN, TST, TEQ

CMP (Compare) subtracts its source operand from the destination register and updates the condition flags — without modifying the destination register. This avoids the corruption of register contents that would occur when using SUBS for comparison:

```

1  CMP R1, R2      ; set flags based on (R1 - R2); R1 unchanged
2  BGE R1geR2      ; branch if R1 >= R2 (signed)
3
4  CMP R1, #4      ; compare R1 with immediate 4
5  BEQ Found       ; branch if R1 == 4

```

Three related conditional test instructions (no S suffix required — they *always* update flags):

- CMN R0, R1 — sets flags based on $R0 + R1$ (Compare Negative).
- TST R0, R1 — sets N, Z, C flags based on $R0$ AND $R1$ (Test Bits).
- TEQ R0, R1 — sets N, Z, C flags based on $R0$ EOR $R1$ (Test Equivalence).

5.4.3 Conditional Execution

In ARM, *any* instruction can be made conditionally executable by appending a condition suffix to its mnemonic. This avoids short forward branches and can reduce code size and pipeline stalls:

```

1  ; C equivalent: if (R0 == 1) { R1 = 3; } else { R1 = 5; }
2  CMP R0, #1      ; set Z = 1 if R0 == 1
3  MOVEQ R1, #3     ; executed only if Z = 1
4  MOVNE R1, #5     ; executed only if Z = 0

```

The conditional execution feature makes the execution of each instruction dependent on the current state of the N, Z, V, C flags, saving the two branch instructions that would otherwise be required.

Key Result: Conditional Execution Syntax

Append any condition code (e.g. EQ, NE, GT) to the mnemonic.

Example: ADDEQ, MOVHI, STRNE.

The S suffix and condition code may be combined: ADDEQS, MOVNES.

5.5 Program Example: Finding the Largest Number (FindMax)

This example integrates data transfer, arithmetic, logical, and program control instructions to find the largest unsigned value in a 10-element integer array starting at address 0x100.

Register allocation:

- R0 — address pointer (current element)
- R1 — loop counter
- R2 — temporary register (current element value)
- R3 — current maximum value (result)

```
1 MOV R0, #0x100      ; pointer to first element
2 MOV R1, #9          ; loop counter (9 more elements after first)
3 LDR R3, [R0]        ; assume first element is the current max
4
5 Loop:
6 ADD R0, R0, #4      ; advance pointer to next element
7 LDR R2, [R0]        ; load next element
8 CMP R2, R3          ; compare R2 with current max R3 (R2 - R3)
9 BLS Skip            ; branch if R2 <= R3 (unsigned: lower or same)
10 MOV R3, R2          ; new maximum found: update R3
11
12 Skip:
13 SUBS R1, R1, #1     ; decrement counter; set Z when R1 = 0
14 BNE Loop            ; loop back while counter != 0
```

Note that BLS (Branch if Lower or Same) is used because the array contains **unsigned** numbers. For a signed array, BLE (Branch if Less than or Equal) would be required instead.

Chapter 6

Modular Programming

6.1 Motivation: Modular Program Design

Real-world software systems are extremely large and complex. Google Chrome contains roughly 6.7 million lines of code, Android OS between 12–15 million, and the Boeing 787 avionics around 6.5 million. No single monolithic function can feasibly represent a program of this scale. Instead, large programs are *decomposed* into several smaller, less complex **modules**, each of which can be designed, tested, and verified independently.

The key advantages of modular design are:

- Modules can be designed and tested **independently**, reducing complexity.
- The same module may be called from multiple places, **reducing overall program size**.
- General-purpose modules can be **reused** across different projects.

Definition 6.1.1 (Good Software Module). A well-designed module exhibits two properties:

1. **Loose coupling** — data within the module is entirely independent of other modules (i.e. it uses local variables).
2. **Strong modularity** — it performs a single, logically coherent task.

Example 6.1.2 (Standard Deviation — Monolithic vs. Modular). Consider computing the standard deviation σ of an array x of N values:

$$\sigma = \sqrt{\frac{\sum_i (x_i - \mu)^2}{N}}$$

Case 1 (monolithic): Everything lives in `main()`, mixing the mean computation and the variance computation in one block.

Case 2 (modular): Two helper functions are defined:

```
1 float sum(int* x, int N) {
2     float tot = 0.0;
3     for (int i = 0; i < N; i++) tot += x[i];
4     return tot;
5 }
6
7 float Sigma_f(int* x, float avg, int N) {
8     float sigma = 0.0;
9     for (int i = 0; i < N; i++) sigma += pow(x[i] - avg, 2);
10    sigma /= N;
11    sigma = sqrt(sigma);
12    return sigma;
13 }
14
15 int main() {
16     float avg = sum(x, N) / N;
```

```

17     float sigma = Sigma_f(x, avg, N);
18     return 0;
19 }

```

The modular version is cleaner, individually testable, and each function can be reused. The question that naturally follows: *how are these C functions translated to ARM assembly, and how does the processor jump to a function and return?*

6.2 Subroutines

In assembly, modules (C functions) are implemented as **subroutines**. A subroutine can be called from multiple locations in a program. The two parties involved are:

- **Caller** — the program that invokes the subroutine.
- **Callee** — the subroutine itself.

Upon completion, a subroutine must return control to the instruction *immediately after* the call in the caller.

6.2.1 Why Not Use Plain B?

Branching with B simply overwrites the Program Counter (PC), so the return address in the calling program is **lost**. One could hardcode a branch back to the caller, but this requires knowing the exact memory address at compile time — impractical in general. The solution is to save the return address automatically before jumping.

6.2.2 Branch with Link (BL)

Definition 6.2.1 (BL — Branch with Link). BL <label> performs two actions atomically:

1. Stores the **return address** (current PC + 4) in the **Link Register** (R14 / LR).
2. Loads the subroutine's address into the PC, transferring control to the subroutine.

The instruction encoding uses a 1-bit field (bit 24) set to 1 to distinguish BL from plain B, along with a 24-bit signed offset.

Execution example: Suppose BL SortSub is at address 0x0014 and SortSub begins at 0x0030. Before execution, PC = 0x0018 (the fetch-ahead value). After execution: LR = 0x0018 (return address), PC = 0x0030 (subroutine start).

Key Result: BL Saves the Return Address

BL SUB1 sets $LR \leftarrow PC + 4$ and jumps to SUB1. The original caller's next-instruction address is preserved in LR so execution can resume there after the subroutine finishes.

6.2.3 Returning: BX LR and MOV PC, LR

Definition 6.2.2 (BX LR — Branch and Exchange). BX LR copies the value in LR back into the PC, returning execution to the instruction after the original BL call.

Warning: VisUAL Does Not Support BX LR

The VisUAL simulator does not support the BX LR instruction. Use MOV PC, LR as an equivalent substitute when working in VisUAL.

6.3 Parameter Passing

Calling programs must supply input data to subroutines and retrieve results. Parameters must be *set up before* the subroutine is called and *cleaned up after* it returns (depending on the passing method). There are three fundamental methods:

1. Via **registers**
2. Via a **memory block**
3. Via the **system stack** (covered in later chapters)

6.3.1 Parameter Passing via Registers

Parameters are loaded into registers before BL is called. This is the simplest and fastest method.

Definition 6.3.1 (ARM Calling Convention — Register Usage). • **R0–R3**: Used to pass argument values to a subroutine, and to return result values to the caller. The subroutine is free to modify these registers.

- **R4–R11**: Used to hold local variables within a subroutine. They are *not* used for argument passing, and **must be preserved** (saved and restored) by the subroutine.
- **R12**: Scratchpad register; does not need to be preserved. Can sometimes be used as an additional return register.
- **Pro**: Efficient — parameters are already in registers and can be used immediately inside the subroutine with no memory access overhead.
- **Con**: Lacks generality — limited to the number of available registers. Unsuitable for subroutines with many parameters (e.g. a `Draw_rectangle(X, Y, Height, Width, Border, Line_style, Fill, Color, Pattern, Shadow)` call with 10 parameters cannot fit in R0–R3 alone).

Exam Tip: Register Calling Convention

By convention, **R0–R3** are argument/return registers (caller-saved). **R4–R11** are callee-saved: if a subroutine uses them, it must push them onto the stack at entry and pop them before returning.

6.3.2 Parameter Passing via Memory Block

A dedicated region of memory acts as a “mailbox” shared between the caller and callee. The caller writes all parameters to consecutive memory locations, then passes the **start address** of this block in an address register (e.g. R0) before calling the subroutine. The subroutine reads from (and writes results back to) that memory region.

- **Pro**: Can pass arbitrarily many parameters, including arrays and structs.
- **Con**: Requires additional memory read/write instructions, so it is slower than register passing.

6.4 Examples

6.4.1 Bit-Counting Subroutine (Passing via Registers)

Task: Count the number of 1-bits in a 32-bit word. The word is passed in R1; the result is returned in R0.

Key instruction — **RRXS**: Rotate Right Extended sets the Carry flag to the bit shifted out, and updates the status register (S suffix). By rotating R1 through the carry 32 times, each bit is tested once.

Solution 1 (using **RRXS** + **ADC**):

```
1 CountIs MOV R0, #0      ; clear result register
2         MOV R2, #32      ; loop counter = 32 bits
3 Loop    RRXS R1, R1      ; rotate R1 right through carry, set flags
4         ADC R0, R0, #0    ; R0 += carry (adds 1 if C=1)
5         SUBS R2, R2, #1   ; decrement counter
6         BNE Loop         ; repeat until all 32 bits tested
7         MOV PC, LR       ; return (equivalent to BX LR)
```

Warning: Carry Flag Corruption

SUBS modifies the Carry flag. On the next loop iteration, **RRXS** will use this corrupted carry value, destroying R1's rotation. This bug is addressed in Solution 2 by avoiding **SUBS** and instead using a counter that does not disturb the carry.

Solution 2 (using **ROR** inline shift + masking, carry-safe):

```
1 CountIs EOR R0, R0, R0    ; R0 = 0
2         ADD R2, R0, #32    ; counter = 32
3         ADD R3, R0, #1     ; mask = 0x00000001
4 Loop    AND R4, R3, R1, ROR R2 ; isolate bit (32-R2) of R1
5         ADD R0, R0, R4     ; accumulate
6         SUBS R2, R2, #1    ; decrement counter
7         BNE Loop
8         MOV PC, LR
```

The inline shift **R1, ROR R2** rotates R1 right by R2 without storing the result, then **AND** with mask 0x1 extracts the current LSB. Because the carry from **SUBS** no longer feeds back into the rotation, Solution 2 avoids the corruption problem.

6.4.2 Lower-to-Upper-Case Subroutine (Passing via Memory)

Task: Convert a NULL-terminated ASCII string (stored byte-by-byte in memory) from lower case to upper case in place. The start address of the string is passed in R0.

Algorithm: For each character, check whether its ASCII value is in [0x61, 0x7A] (i.e. 'a' to 'z'). If so, subtract 32 to obtain the uppercase equivalent (e.g. 'a' = 0x61 → 'A' = 0x41). Stop when the NULL byte (0x00) is encountered.

Memory layout: With byte-efficient storage, characters occupy consecutive single bytes (e.g. "apple" at addresses 0x100–0x104, followed by 0x00 at 0x105). Use **LDRB**/**STRB** to load/store one byte at a time.

```
1 Lo2Up  STMFD SP!, {r0, r1} ; save R0 and R1 on stack
2
3 Loop    LDRB R1, [R0], #1    ; load byte at [R0], then R0 += 1
4         CMP R1, #0          ; check for NULL terminator
5         BEQ Done            ; if NULL, we are done
6         CMP R1, #0x061      ; compare with 'a'
```

```

7      BLT   Loop           ; below 'a' -> skip
8      CMP   R1, #0x07A     ; compare with 'z'
9      BGT   Loop           ; above 'z' -> skip
10     SUB   R1, R1, #32     ; convert to uppercase
11     STRB  R1, [R0, #-1]   ; write back to original address
12
13     B      Loop           ; next character
14
15 Done  LDMFD SP!, {r0, r1} ; restore R0 and R1
16      MOV   PC, LR        ; return

```

Remark 6.4.1 (Transparent Subroutine). The subroutine modifies R0 and R1 internally but restores them via the stack push/pop (STMFD/LDMFD) before returning. This makes it a **transparent subroutine**: the caller’s register state is completely preserved, so the subroutine can be inserted anywhere without side-effects.

Exam Tip: Post-Indexed Addressing with LDRB

LDRB R1, [R0], #1 loads the byte at address R0 into R1, *then* increments R0 by 1. To write back to the *original* address, use STRB R1, [R0, #-1] (base + offset -1, without updating R0). This pattern — post-increment on load, negative offset on store — is idiomatic for in-place string processing.

6.5 The System Stack

The **system stack** is a first-in, last-out (FILO) linear data structure maintained in the memory’s data area. In the ARM processor it is managed by the dedicated **Stack Pointer** (SP, also called R13), which always points to the top item on the stack.

The stack can grow towards either lower or higher memory addresses. The preferred convention is to grow towards *lower* addresses, starting from the highest available address. This way the stack and the heap/code regions grow towards each other, maximising the usable address space before a collision occurs.

The three fundamental stack operations are **push** (write a new item on top), **pop** (remove and read the top item), and **access** (read an item at a given offset from the current SP without changing SP).

Remark 6.5.1. After a pop, the data is *not* erased from memory; it is simply no longer within the “live” region of the stack. The bytes remain at their physical address and can still be read with a base+offset load (e.g. LDR R0, [SP, #-4]) as long as SP has not been advanced past them by a subsequent push.

6.5.1 Push and Pop Instructions

Push (single register). Pushing a register onto a full-descending stack requires the SP to be *pre-decremented* before the store. This is achieved with the pre-indexed write-back form of STR:

```

1  STR R0, [SP, #-4]! ; SP = SP - 4, then Mem[SP] = R0

```

Pop (single register). Popping restores a register and *post-increments* SP:

```

1  LDR R0, [SP], #4 ; R0 = Mem[SP], then SP = SP + 4

```

Push/pop multiple registers. ARM provides dedicated block-transfer instructions for pushing and popping several registers in a single instruction:

```
1 STMFD SP!, {list} ; push: store list in descending order, update SP
2 LDMFD SP!, {list} ; pop: load list in ascending order, update SP
```

STMFD stands for *Store Multiple, Full Descending*. Registers in the list are stored highest-numbered first (descending address), so that after an LDMFD with the same list each register is restored to its original value. The ! suffix writes the updated SP back.

Example 6.5.2 (Pushing and Popping Two Registers).

```
1 STMFD SP!, {R1, R0} ; push R1 then R0 (R1 at higher address)
2 ; ... subroutine body ...
3 LDMFD SP!, {R1, R0} ; pop R1 then R0, restoring both
```

After STMFD SP!, {R1,R0} with SP initially at 0xF00: SP moves to 0xEF8; the word at 0xEF8 holds R0's value and the word at 0xEFC holds R1's value.

6.6 Parameter Passing via the Stack

Passing parameters through the system stack is the most general method. It requires no reserved registers, supports an arbitrary number of parameters, and—crucially—enables **recursive** programming because each call level gets its own copy of the parameters. The only constraint is that the stack must not overflow.

The protocol is:

1. The calling program pushes the parameters onto the stack.
2. BL <subroutine> is issued (SP now points to the pushed parameters, and LR holds the return address).
3. The subroutine retrieves its parameters from the stack using SP-indirect with offset addressing.
4. On return, the calling program *immediately* removes the parameters from the stack, typically by adding $4 \times (\text{number of parameters})$ to SP.

Warning: Stack Cleanup is the Caller's Responsibility

Parameters pushed before a BL must be removed by the calling program *immediately after* returning. Failing to do so causes repeated pushes to eventually exhaust the stack space (stack overflow).

Example 6.6.1 (Sum from 1 to N via Stack Parameters). Compute $\sum_{k=1}^N k$ for $N = 5$ and store the result at the memory variable **Answer** (address 0x100). Both parameters—the value N and the address of **Answer**—are passed via the stack.

Calling program:

```
1 MOV R1, #5 ; N = 5 (pass by value)
2 MOV R0, #0x100 ; address of Answer (pass by reference)
3 STMFD SP!, {R1, R0} ; push both parameters
4 BL Sum1N ; call subroutine
5 ADD SP, SP, #8 ; remove two 4-byte parameters from stack
```

Subroutine Sum1N:

```

1 Sum1N  STMFD SP!, {R4, R5, R6} ; save callee-saved registers
2        LDR  R5, [SP, #16]      ; retrieve N      (SP+16)
3        LDR  R6, [SP, #12]      ; retrieve &Answer (SP+12)
4        MOV  R4, #0             ; initialise sum = 0
5 Loop   ADD  R4, R4, R5          ; sum += current N
6        SUBS R5, R5, #1          ; N-- (sets flags)
7        BNE  Loop               ; repeat until N = 0
8        STR  R4, [R6]            ; write sum to Answer
9        LDMFD SP!, {R4, R5, R6} ; restore callee-saved registers
10       MOV  PC, LR              ; return

```

Stack layout after STMFD SP!, {R4,R5,R6} inside the subroutine (offsets measured from SP at that point):

SP+0	Saved R4
SP+4	Saved R5
SP+8	Saved R6
SP+12	Address of Answer
SP+16	$N = 5$

Hence LDR R5, [SP, #16] retrieves N and LDR R6, [SP, #12] retrieves the address of Answer.

Key Result: SP-Offset Arithmetic for Stack Parameters

When a subroutine saves k callee registers via STMFD SP!, {...} on entry, the first caller-pushed parameter is at [SP, #4 k] and subsequent ones at [SP, #4($k-1$)], [SP, #4($k-2$)], ... You must account for every word pushed *after* the parameters in order to compute the correct offset.

6.7 Transparent Subroutines (Revisited)

Recall from Section 6.4.2 that a *transparent* subroutine preserves all CPU registers visible to the caller. The formal mechanism is:

1. On entry, save every *callee-saved* register the subroutine will modify (R4–R11) onto the stack using STMFD.
2. Perform the subroutine's work using those registers freely.
3. Before returning, restore all saved registers from the stack using LDMFD.

```

1 SUB1   STMFD SP!, {R4-R7} ; save R4--R7 on entry
2        ; ... body using R4--R7 ...
3        LDMFD SP!, {R4-R7} ; restore R4--R7 before return
4        MOV  PC, LR        ; return to caller

```

Registers R0–R3 are *caller-saved* by the ARM calling convention (see Definition 6.3.1): the caller must save them if their values are needed after the call.

6.8 Passing by Value and by Reference

Parameters can be passed in two fundamental ways:

Definition 6.8.1 (Pass by Value). The *actual data value* is copied into the parameter slot (register or stack word). The subroutine operates on its own copy; changes do not propagate back to the caller.

Definition 6.8.2 (Pass by Reference). The *address* of the data is passed instead of the data itself. The subroutine dereferences this address to read or write the original variable.

Pass by reference is preferred when:

- The subroutine needs to **modify** the caller’s variable (e.g. writing a result back to **Answer**).
- A **large data structure** (e.g. an array) must be shared—copying it entirely would be wasteful.

In the Sum1N example (Example 6.6.1), *N* is passed *by value* (its integer value 5 is pushed) while **Answer** is passed *by reference* (the address 0x100 is pushed so the subroutine can write the result directly).

Example 6.8.3 (C Mean Function: Mixed Passing).

```

1 int Mean(int *Array, int N) // Array: by reference; N: by value
2 {
3     int avg, i, sum = 0;    // local variables
4     for (i = 0; i < N; i++)
5         sum += Array[i];
6     avg = sum / N;
7     return avg;            // single return value -> R0 by convention
8 }

```

Array is a pointer (pass by reference) so the subroutine can traverse the array without copying it. *N* is a plain integer (pass by value). The single return value **avg** is placed in R0 by the ARM calling convention.

6.9 Local Variables and the Stack Frame

Subroutines often need **local variables** whose scope and lifetime are confined to a single invocation. (This is an instance of *loose coupling*—the module’s internal data is entirely hidden from callers.) Because local variables must be created on entry and destroyed on exit, the system stack is the natural home for them. The reserved stack region allocated for a subroutine’s local variables is called its **stack frame**.

Definition 6.9.1 (Stack Frame). A *stack frame* (or *activation record*) is a contiguous block of memory allocated on the system stack immediately upon entry into a subroutine. It holds the subroutine’s local variables and, optionally, saved registers and incoming parameters. The frame is released when the subroutine returns.

A stack frame of *N* words is created by subtracting $4N$ from SP on entry:

```

1 SUB SP, SP, #(4*N) ; allocate N word-sized local variable slots

```

and released by adding $4N$ back to SP before returning.

6.9.1 Accessing the Stack Frame via the Frame Pointer

One approach is to designate a register as a **frame pointer** (FP). By ARM convention, R11 is used as FP.

1. Save the caller’s R11 on the stack.
2. Set R11 = SP (FP now points to the saved R11 word).
3. Allocate the frame by subtracting $4N$ from SP.

Any local variable at slot i ($i = 1, 2, \dots, N$) is then accessed as $[R11, \#-4i]$. For example, with three local variables `avg`, `i`, `sum`:

$\text{avg} \leftrightarrow [R11, \#-4], \quad i \leftrightarrow [R11, \#-8], \quad \text{sum} \leftrightarrow [R11, \#-12]$

On exit, the frame is destroyed by restoring the original SP and R11:

```
1 ADD SP, SP, #16 ; for N=3: 3 locals (12 bytes) + saved R11 (4 bytes)
2 LDR R11, [R11] ; restore caller's R11 from saved location
```

6.9.2 Accessing the Stack Frame via the Stack Pointer

A simpler, more efficient alternative is to access local variables directly through SP using positive offsets:

1. Allocate the frame: `SUB SP, SP, #(4N)`.
2. Access slot i as `[SP, #4(i-1)]`.

$\text{avg} \leftrightarrow [SP, \#0], \quad i \leftrightarrow [SP, \#4], \quad \text{sum} \leftrightarrow [SP, \#8]$

Advantage: No extra register (FP) is needed.

Disadvantage: The stack cannot be used for further pushes inside the subroutine without adjusting all offsets, since SP is the reference point.

6.10 Nested Subroutines

A **nested subroutine** occurs when a subroutine (the callee) itself calls another subroutine. In contrast to the simple, single-level calls seen earlier — where the main program calls one subroutine, which returns directly — nested calls introduce a critical problem: the Link Register (LR) is overwritten each time BL executes.

6.10.1 The LR Overwrite Problem

Consider a main program at address `0x400` that calls `DotProd` via BL `Sub1`. At that moment, LR is set to `0x404` (the return address). `DotProd` then calls `Mult` via BL `Sub2`. This second BL overwrites LR with the new return address (`0xA44`) — the original value `0x404` is *lost*. When `Mult` returns, execution correctly arrives back inside `DotProd`; however, when `DotProd` later executes `BX LR`, it branches to `0xA44` again instead of `0x404`, causing the program to get stuck inside `DotProd` forever.

Warning: LR is Overwritten by Every BL

Each BL instruction *unconditionally* replaces the current value of LR with `PC + 4`. Any subroutine that itself issues a BL must preserve LR before doing so, or the original return address is permanently lost.

6.10.2 Solution: Saving LR onto the System Stack

The solution is to push LR onto the system stack at the *beginning* of every subroutine that makes further BL calls. This is typically done by including LR in the `STMFD` register list:

```

1 MySub  STMFD  SP!, {R4-R7, LR} ; push callee-saved regs + LR
2        ; ... body that may call BL SomeOther ...
3        LDMFD  SP!, {R4-R7, PC} ; restore regs; PC <- saved LR (returns)

```

The final LDMFD pops the saved LR value directly into **PC**, which simultaneously restores the general-purpose registers *and* performs the return. This is equivalent to LDMFD SP!, {R4-R7, LR} followed by MOV PC, LR, but executed as a single instruction.

There are two valid strategies for saving LR:

1. Save just before each BL:

```

1  STR  LR, [SP, #-4]! ; push LR
2  BL   SubRoutine
3  LDR  LR, [SP], #4   ; pop LR

```

Advantage: No impact on other stack offsets. *Disadvantage:* Uses a different instruction structure from the standard STMFD/LDMFD pattern.

2. Include LR in the entry STMFD (preferred):

```

1 MySub  STMFD  SP!, {R4-R7, LR}
2        ; ... BL calls here ...
3        LDMFD  SP!, {R4-R7, PC}

```

Advantage: Consistent style — one push/pop pair covers the whole subroutine. *Disadvantage:* Adding LR to the register list shifts all subsequent stack offsets by 4 bytes.

Key Result: Stack Offset Adjustment When Saving LR

When LR is added to the STMFD push list, *all* stack-relative offsets used to access caller-passed parameters must increase by 4. For example, if 4 registers (R4-R7) were previously saved, parameters at [SP, #16], [SP, #20], ... become [SP, #20], [SP, #24], ... once LR is also pushed.

6.10.3 Worked Example: Dot-Product with Nested Multiply

The dot-product $Z = \sum_{i=0}^{N-1} X_i \times Y_i$ is computed by DotProd, which calls a helper subroutine Mult. Parameters (base addresses of X and Y, size N, and destination address Z) are passed via the system stack. Mult receives two operands in R0 and R1, and returns the product in R12.

Multiply subroutine (leaf — no nested calls):

```

1 Mult   MOV    R12, #0      ; Clear accumulator
2 Loop   ADD    R12, R12, R0  ; R12 += R0
3        SUBS   R1, R1, #1    ; R1-- (set flags)
4        BNE    Loop         ; loop while R1 != 0
5        MOV    PC, LR       ; return (BX LR)

```

Because Mult makes no further BL calls, it does not need to save LR.

DotProd subroutine (nested — calls Mult): When DotProd is entered the stack already holds (from top to bottom): destination address (0x300), N (20), base of Y (0x200), base of X (0x100). After STMFD SP!, {R4-R7, LR} pushes five registers (20 bytes), the parameter offsets are:

[SP, #20] destination address
 [SP, #24] *N*
 [SP, #28] base of *Y*
 [SP, #32] base of *X*

```

1 DotProd STMFD SP!, {R4-R7, LR} ; save regs + LR
2
3     LDR    R4, [SP, #32]      ; R4 = base address of X
4     LDR    R5, [SP, #28]      ; R5 = base address of Y
5     LDR    R6, [SP, #24]      ; R6 = N (loop counter)
6     MOV    R7, #0             ; R7 = accumulator (sum = 0)
7
8 Loop1 LDR    R0, [R4], #4      ; R0 = X[i]; post-increment R4 by 4
9       LDR    R1, [R5], #4      ; R1 = Y[i]; post-increment R5 by 4
10      BL     Mult              ; R12 = X[i] * Y[i]
11      ADD    R7, R7, R12       ; sum += product
12      SUBS   R6, R6, #1        ; N--
13      BNE    Loop1            ; repeat until N == 0
14
15      LDR    R4, [SP, #20]      ; R4 = destination address
16      STR    R7, [R4]          ; store final sum
17
18      LDMFD  SP!, {R4-R7, PC}  ; restore and return

```

Exam Tip: Tracing Stack Offsets

When computing stack offsets after an **STMFD**, count the number of registers in the list and multiply by 4 to find the total SP shift. Each caller-pushed parameter is then accessed at that shift plus the parameter's original distance above the pre-**STMFD** stack top.

6.11 Recursive Subroutines

A **recursive subroutine** is one that calls itself from within its own body. Recursion is a powerful technique for problems with systematic, self-similar structure — such as computing factorials, Fibonacci numbers, or performing tree traversals.

6.11.1 Requirements for Correct Recursion

Two conditions must both be satisfied:

1. **Base case (stopping condition):** A condition under which the subroutine does *not* make a recursive call. Without this, the recursion is infinite and the stack overflows.
2. **Return address preservation:** LR must be saved onto the stack at the entry of every level of the recursion, because each **BL Recur** overwrites LR with a new return address.

Warning: Infinite Recursion Without a Base Case

If no stopping condition is present, the subroutine calls itself indefinitely, consuming stack space with each call until a hardware or memory fault occurs.

The general template for a recursive ARM subroutine is:

```

1 Recur STMFD SP!, {..., LR} ; save registers including LR
2
3     ; --- base case check ---

```

```

4      ; if base case met: set result, branch to Done
5
6      ; --- recursive step ---
7      ; prepare argument for recursive call
8      BL      Recur          ; call self
9
10     ; --- post-recursion combination ---
11
12 Done  LDMFD   SP!, {..., LR}
13      MOV     PC, LR          ; return (use instead of BX LR in VisUAL)

```

6.11.2 Worked Example: Fibonacci Sequence

The Fibonacci sequence is defined recursively as:

$$F(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ F(n-1) + F(n-2) & n > 2 \end{cases}$$

producing 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

The parameter n is passed via the top of the stack; the result is returned in R12.

C pseudocode:

```

1  int Fibonacci(int n) {
2      if (n == 1) return 0;
3      if (n == 2) return 1;
4      return Fibonacci(n - 1) + Fibonacci(n - 2);
5  }

```

ARM assembly implementation: After STMFD SP!, {R4-R6, LR} pushes four registers (16 bytes), the parameter n originally at the stack top is now at [SP, #16].

```

1  Fib      STMFD   SP!, {R4-R6, LR} ; save R4, R5, R6, LR (SP -= 16)
2
3          LDR      R4, [SP, #16]    ; R4 = n
4
5          ; --- base cases ---
6          CMP      R4, #1
7          MOVEQ    R12, #0          ; F(1) = 0
8          CMP      R4, #2
9          MOVEQ    R12, #1          ; F(2) = 1
10         BLE      Done             ; n <= 2: result ready, skip recursion
11
12         ; --- compute F(n-1) ---
13         SUB      R5, R4, #1        ; R5 = n-1
14         STMFD    SP!, {R5}        ; push n-1 as argument for Fib
15         BL       Fib              ; R12 = F(n-1)
16         LDMFD    SP!, {R5}        ; pop argument
17
18         MOV      R6, R12           ; R6 = F(n-1) (preserve before next call)
19
20         ; --- compute F(n-2) ---
21         SUB      R5, R4, #2        ; R5 = n-2
22         STMFD    SP!, {R5}        ; push n-2 as argument for Fib
23         BL       Fib              ; R12 = F(n-2)
24         LDMFD    SP!, {R5}        ; pop argument
25

```

```

26      ADD    R12, R12, R6      ; R12 = F(n-1) + F(n-2)
27
28 Done  LDMFD  SP!, {R4-R6, LR} ; restore registers
29      MOV    PC, LR           ; return

```

Key observations:

- LR is saved in the entry `STMFD` so that each recursive `BL Fib` does not corrupt the caller's return address.
- After computing $F(n-1)$, the result in R12 is moved to R6 *before* the second recursive call, because that call will overwrite R12.
- The argument is pushed onto the stack immediately before each `BL` and popped immediately after, keeping the stack balanced at each level.
- R4 (holding n) survives both recursive calls because it was saved in the initial `STMFD` and the callee (`Fib` itself) also saves R4.

Key Result: Rules for Recursive ARM Subroutines

1. Always include LR in the entry `STMFD` push list.
2. Define a base case that branches past any recursive `BL`.
3. Save intermediate results in a callee-saved register before the next recursive call overwrites R12.
4. Push each argument immediately before its `BL` and pop it immediately after to maintain a balanced stack.

6.12 Chapter Summary

- `BL <label>` calls a subroutine: it saves `PC+4` into LR and jumps to the label. `BX LR` (or `MOV PC, LR` in VisUAL) returns by copying LR back to PC.
- **Register passing** (R0–R3 for arguments and return values) is the simplest and fastest method, but is limited by the number of available registers.
- **Memory-block passing** and **stack passing** support large or complex parameter sets; stack passing is the most general and is used by compiled C code.
- Subroutines that modify R4–R11 must save and restore them (`STMFD/LDMFD`) to remain caller-transparent.
- **Nested subroutines** arise when a subroutine itself issues a `BL`. The LR is overwritten by every `BL`, so it must be pushed onto the stack at the start of any subroutine that makes further calls: `STMFD SP!, {..., LR}`. Adding LR to the push list shifts all stack-relative parameter offsets by 4 bytes.
- **Recursive subroutines** call themselves. Two requirements must be met: (1) a **base case** that stops the recursion, and (2) LR saved on the stack at every recursive level. Intermediate results must be preserved in callee-saved registers before the next recursive `BL` overwrites R12.

Chapter 7

Flow-Control Constructs

7.1 IF Statements

The **IF** construct is one of the most fundamental flow-control structures in high-level languages. When translating it to assembly, a naïve approach is to *imitate* the high-level test condition directly:

```
1 ; C: if (a > b) { S1 }
2     CMP a, b
3     BGT DoIF      ; branch if a > b
4     B Skip        ; unconditional jump over S1
5 DoIF {S1}
6 Skip
```

This approach requires *two* branch instructions. A more efficient strategy is to **reverse the test condition**, allowing the unconditional branch to be eliminated:

```
1 ; C: if (a > b) { S1 }
2     CMP a, b
3     BLE Skip      ; branch if NOT (a > b), i.e. reverse condition
4     {S1}
5 Skip
```

Key Result: Reversing the IF Condition

To implement an `if (cond) {S1}` construct efficiently, *reverse* the condition and branch over `S1`. This eliminates the extra unconditional branch and saves one instruction per IF statement.

Common reverse pairs: `HS ↔ LO`, `LT ↔ GE`, `EQ ↔ NE`, `GT ↔ LE`.

7.1.1 Example: Find Largest Number

As a practical application, consider a loop that scans an array to find the maximum value. The inner IF logic — updating the current maximum when a larger element is found — is implemented using the reversed-condition technique:

```
1 ; R0 = pointer to current array element
2 ; R1 = loop counter
3 ; R2 = current element (temporary)
4 ; R3 = current maximum value (result)
5
6     MOV R0, #0x100    ; pointer to first element
7     MOV R1, #9        ; counter = 9 (nine more elements to check)
8     LDR R3, [R0]      ; assume first element is current max
9
10 Loop
11     LDR R2, [R0, #4]! ; load next element, auto-increment pointer
12     CMP R2, R3        ; compare: R2 - R3
```

```

13     BLO Skip          ; if R2 < R3, skip update
14     MOV R3, R2        ; update max: R3 = R2
15
16 Skip
17     SUBS R1, R1, #1    ; decrement counter, set flags
18     BNE Loop          ; loop while counter != 0

```

Warning: Avoiding Reversion Introduces Redundancy

Attempting to use the *non-reversed* condition (branching *into* the body rather than over it) requires duplicating the loop-control instructions (SUBS, BNE) after the IF body, creating redundant code. The reversed-condition approach avoids this entirely.

7.2 Conditional Execution in ARM

The 32-bit ARM ISA extends the concept introduced in Chapter 5 (see Section 5.4.3) by allowing *any* instruction — not just branches — to be conditionally executed based on the current Condition Code (CC) flags. This is a powerful feature that can eliminate branch instructions entirely.

Consider the simple IF:

```

1 ; C: if (r0 == 1) r1 = 3;
2
3 ; --- With branch ---
4     CMP r0, #1
5     BNE Skip
6     MOV r1, #3
7 Skip
8
9 ; --- With conditional execution ---
10    CMP r0, #1
11    MOVEQ r1, #3 ; MOV executes only if EQ flag is set
12 Skip

```

The conditional execution version uses one fewer instruction and — crucially — does not alter the flow of the program counter, keeping the CPU pipeline intact.

Exam Tip: Conditional Execution Across Multiple Instructions

Conditional execution is not limited to a single instruction. It remains valid for *any number of consecutive instructions*, provided no intervening instruction updates the CC flags (i.e. no instruction with the S suffix, and no new CMP/TST between the comparison and the conditionally-executed code).

7.2.1 Find Largest with Conditional Execution

Applying conditional execution to the FindMax example eliminates the branch entirely inside the loop, yielding a significantly smaller and faster inner loop:

```

1     MOV R0, #0x100
2     MOV R1, #9
3     LDR R3, [R0]
4
5 Loop
6     LDR R2, [R0, #4]!
7     CMP R2, R3

```

```

8  MOVGE R3, R2      ; update max only if R2 >= R3 (no branch!)
9  SUBS  R1, R1, #1
10 BNE   Loop

```

7.2.2 Find Largest Number and Store Its Index

When the task requires storing both the maximum value *and* its index, two updates must be performed conditionally. Without conditional execution, a branch is unavoidable:

```

1  ; R4 = index of current maximum
2
3  MOV  R0, #0x100
4  MOV  R1, #9
5  MOV  R4, #0
6  LDR  R3, [R0]
7
8  Loop
9      LDR  R2, [R0, #4]!
10     CMP  R2, R3
11     BLO  Skip
12     MOV  R3, R2      ; update max value
13     RSUB R4, R1, #10 ; update index: index = 10 - counter
14
15  Skip
16     SUBS R1, R1, #1
17     BNE  Loop

```

With conditional execution, both updates are made inline without any branch:

```

1  Loop
2      LDR  R2, [R0, #4]!
3      CMP  R2, R3
4      MOVGE R3, R2      ; conditional update of max
5      RSUBGE R4, R1, #10 ; conditional update of index
6      SUBS R1, R1, #1
7      BNE  Loop

```

Both MOVGE and RSUBGE execute only when $R2 \geq R3$. Because neither carries the S suffix, the CC flags set by CMP are preserved across both conditional instructions.

7.3 IF-ELSE Statements

The IF-ELSE construct requires both a conditional and an unconditional branch when implemented without conditional execution:

```

1  ; C: if (a == 3) { S1 } else { S2 }
2
3      CMP  a, #3
4      BEQ  DoIf
5      {S2}      ; ELSE body
6      B    Skip
7  DoIf
8      {S1}      ; IF body
9  Skip

```

Remark 7.3.1. Reversing the test condition in an IF-ELSE construct does *not* improve efficiency in general — both branches always execute one conditional and one unconditional jump. However, reversing is beneficial when the ELSE code segment $\{S2\}$ is *more likely to execute*, because the

unconditional branch (which takes slightly longer on a pipelined CPU) is then encountered less frequently.

7.3.1 Conditional Execution for IF-ELSE

When each branch body is a single instruction, conditional execution removes *all* branch instructions, reducing the IF-ELSE to exactly two instructions:

```

1 ; C: if (r0 == 1) r1 = 3; else r1 = 5;
2
3     CMP    r0, #1
4     MOVEQ  r1, #3    ; execute if r0 == 1
5     MOVNE  r1, #5    ; execute if r0 != 1

```

In this form, *Skip* is always the *third* instruction regardless of the branch outcome, which means the CPU pipeline is never flushed.

7.4 Compound Conditions

7.4.1 Compound AND Conditions

A compound AND condition such as `if ((a == b) && (b > 0)) {S1}` is resolved by a compiler into a *series of independent simple tests*:

```

1     if (a != b) then Skip    ; reverse of a == b
2     if (b <= 0) then Skip    ; reverse of b > 0
3     {S1}
4     Skip

```

The tests are evaluated **left to right**. As soon as the first condition evaluates to *false*, execution jumps to *Skip* without testing the remaining conditions. This optimisation is known as **fast Boolean evaluation** (or short-circuit evaluation).

Key Result: Optimising Compound AND

Place the **least likely** condition **leftmost** in a compound AND expression. Because a false result short-circuits the evaluation, the unlikely condition acts as an early-exit gate, avoiding the cost of evaluating the remaining (more expensive) conditions.

Warning: Compound AND with Conditional Execution

It may be tempting to use chained conditional comparisons for a compound AND:

```

1     CMP    R1, R2          ; R1 <- a, R2 <- b
2     CMPEQ  R2, #0          ; only compare b to 0 if a == b
3     XXXGT  XXXX            ; execute S1 if b > 0

```

This is *incorrect* when $a > b$ and $b > 0$: the first `CMP` sets a non-EQ flag, so `CMPEQ` does not execute (CC flags retain the result of the first comparison), and `XXXGT` fires incorrectly. Always verify that chained conditional comparisons cover all cases before using them.

7.4.2 Compound OR Conditions

A compound OR such as `if ((a == 1) || (a == 2)) {S1}` requires a different strategy. The goal is to *branch into* the IF body as soon as any condition is true:

```

1   if (a == 1) then DoIf
2   if (a != 2) then Skip ; reversed last condition avoids extra B
3 DoIf
4   {S1}
5 Skip

```

Most compilers eliminate an unconditional jump at the end of the OR series by *reversing the final conditional test*, as shown above.

Key Result: Optimising Compound OR

Place the **most likely** condition **leftmost** in a compound OR expression. Since any true result short-circuits evaluation, the most probable condition will trigger the early exit most often, minimising total instruction count.

An alternative using conditional execution duplicates the body S1 but avoids branches entirely when S1 is a single instruction:

```

1   CMP R1, #1      ; R1 <- a
2   XXREQ XXXX      ; S1 if a == 1
3   CMPNE R1, #2    ; only test a == 2 if previous test was false
4   XXREQ XXXX      ; S1 if a == 2

```

For multi-instruction bodies, the holistic branch approach is preferred:

```

1   CMP R1, #1
2   BEQ doIF
3   CMP R1, #2
4 doIF
5   XXREQ XXXX ; S1 (enters here either directly or from BEQ)

```

7.5 Branchless Logic

Branch instructions (Bcc) can be costly on pipelined processors: if the CPU pre-fetches the wrong next instruction, the pipeline must be **flushed**, wasting several cycles. **Branchless logic** avoids conditional jumps altogether by exploiting arithmetic or bitwise relationships to compute the desired Boolean outcome directly.

Definition 7.5.1 (Branchless Logic). A coding technique that transforms a conditional assignment into a sequence of arithmetic/bitwise instructions with no branch, relying on the algebraic relationship between the test condition and the required output value.

Example 7.5.2. Consider the following construct:

```

1   if ((X & 2) == 2)
2       X = true; // i.e. X = 1
3   else
4       X = false; // i.e. X = 0

```

The bit at position 1 of *X* is already the Boolean answer. Masking it and rotating it into bit position 0 produces the result without any branch:

```

1   AND R1, R1, #0x02 ; isolate bit 1: 0000...0010 (if set)
2   ROR R1, R1, #1    ; rotate right 1: 0000...0001 (true) or 0000...0000 (false)

```

The ROR shifts bit 1 into bit 0 (the LSB), giving the correct Boolean value.

Exam Tip: Conditional Execution as Branchless Code

Conditional execution instructions (e.g. `MOVEQ`, `MOVGE`) are themselves a form of branchless logic — the program counter never branches, so the pipeline is never flushed. When the IF body is short (1–2 instructions), always prefer conditional execution over a `Bcc` branch.

7.6 SWITCH Statements

The SWITCH construct allows a variable to select from multiple execution paths. The assembly code generated by a compiler varies depending on the *nature and range* of the case values. Two scenarios are considered:

Definition 7.6.1 (Jump Table). A data structure stored in code memory containing a list of start addresses for each case’s code segment. The switch variable acts as an index into the table; the corresponding address is loaded directly into the PC to dispatch execution.

7.6.1 Consecutive Narrow-Range Values — Jump Table

When case values are consecutive integers spanning a narrow range (e.g. 0, 1, 2, 3), a jump table is used to dispatch execution in $O(1)$ time, regardless of the number of cases:

```
1  LDR R1, [R2]          ; load switch variable x
2  ADR R3, TBL           ; R3 = base address of jump table
3  LDR PC, [R3, R1, LSL#2] ; PC = TBL[x * 4] (each entry is 4 bytes)
4
5  RES STR R0, [R2, #4]   ; common result store (break target)
6  ...
7
8  C1 MOV R0, #70 ; 'F'
9    B  RES
10 C2 MOV R0, #68 ; 'D'
11    B  RES
12 C3 MOV R0, #67 ; 'C'
13    B  RES
14
15 ; Jump table (in data/code memory)
16 TBL DCD C1      ; TBL + 0 -> 0xA00 (address of C1)
17    DCD C2      ; TBL + 4 -> 0xA08
18    DCD C3      ; TBL + 8 -> 0xA10
```

Key Result: Jump Table Properties

A jump table provides $O(1)$ dispatch: execution time is constant and independent of the number of cases. This is superior to a cascaded if-else-if chain, which requires $O(n)$ comparisons in the worst case.

When the switch includes a **default** case, the compiler pre-loads the default result and guards the jump table with a range check before indexing into it:

```
1  LDR R1, [R2]
2  MOV R0, #88 ; 'X' = default value, pre-loaded
3  CMP R1, #3  ; check upper bound of valid range
4  BGT RES     ; if x > 3, skip table (take default)
5  ADR R3, TBL
6  LDR PC, [R3, R1, LSL#2]
7  RES STR R0, [R2, #4]
```

When the body of each case is a single assignment, conditional execution can replace the jump table and branches entirely:

```

1  LDR  R1, [R2]
2  CMP  R1, #0 ; MOVEQ R0, #70 ; 'F'
3  CMP  R1, #1 ; MOVEQ R0, #68 ; 'D'
4  CMP  R1, #2 ; MOVEQ R0, #67 ; 'C'
5  CMP  R1, #3 ; MOVEQ R0, #66 ; 'B'
6  RES STR R0, [R2, #4]

```

7.6.2 Random Wide-Range Values — Forked if-else-if

When case values span a wide range (e.g. 1, 10, 100, 1000), a jump table would require thousands of entries for the gaps between values, making it impractical. A cascade of if-else-if comparisons is used instead. To reduce the average number of comparisons, a **forked if-else-if** (binary partition) algorithm is applied:

```

1  // Standard (linear) cascade -- O(n) worst case
2  if (x == 1) { S0 }
3  else if (x == 10) { S1 }
4  else if (x == 100) { S2 }
5  else if (x == 1000) { S3 }
6
7  // Forked (binary partition) -- O(log n) average
8  if (x <= 10) {
9      if (x == 1) { S0 }
10     else if (x == 10) { S1 }
11 } else {
12     if (x == 100) { S2 }
13     else if (x == 1000) { S3 }
14 }

```

Exam Tip: Choosing Between Jump Table and Forked if-else-if

Use a **jump table** when case values are consecutive and span a narrow range — lookup is $O(1)$. Use a **forked if-else-if** when values are random or widely spread — the jump table would be too large and mostly empty.

7.6.3 Extended Example: Grade Statistics Subroutine

The following example combines a DO-WHILE loop, a SWITCH construct with a jump table, and parameter passing via the stack (see Section 6.6).

The subroutine `Cal_grades` accepts a null-terminated character array of grades ('A'–'D') and returns three counters: number of top grades (A and D), B-grades, and other passing grades (C).

```

1  void Cal_grades(char* arr, int* top, int* Bgrading, int* other) {
2      *top = 0; *Bgrading = 0; *other = 0;
3      int index = 0;
4      do {
5          char val = arr[index++];
6          switch (val) {
7              case 'A': (*top)++; break;
8              case 'B': (*Bgrading)++; break;
9              case 'C': (*other)++; break;
10             case 'D': (*top)++; break;
11             default: return; // null terminator -> done

```

```

12     }
13     } while (1);
14 }

```

The ARM assembly implementation uses a jump table dispatched via `LDR PC, [R9,R7,LSL#2]`, where R7 is the ASCII code of the grade shifted by subtracting 65 ('A'), turning it into a zero-based table index:

```

1 ; Register allocation:
2 ;   R4 = top counter, R5 = Bgrading counter, R6 = other counter
3 ;   R7 = current character (grade), R8 = scratch pointer
4 ;   R9 = base address of jump table (Base)
5 ;   Stack (caller-pushed): arr addr @ [SP+36], top* @ [SP+32],
6 ;                           Bgrading* @ [SP+28], other* @ [SP+24]
7
8 Cal_grades
9     STMFD SP!, {R4-R9}      ; save registers
10    ADR    R9, Base          ; R9 = jump table base address
11    MOV    R4, #0            ; initialise counters
12    MOV    R5, #0
13    MOV    R6, #0
14    LDR    R8, [SP, #36]     ; R8 = address of grade array
15
16 Loop
17     LDRB   R7, [R8], #1      ; load next byte, auto-increment
18     CMP    R7, #0           ; null terminator?
19     BEQ    Done             ; yes -> exit loop
20     SUB    R7, R7, #65       ; convert ASCII to 0-based index
21     LDR    PC, [R9, R7, LSL#2]; dispatch via jump table
22
23 Return
24     B      Loop             ; back to next grade
25
26 Done
27     LDR    R8, [SP, #32]     ; store results via pointers
28     STR    R4, [R8]
29     LDR    R8, [SP, #28]
30     STR    R5, [R8]
31     LDR    R8, [SP, #24]
32     STR    R6, [R8]
33     LDMFD  SP!, {R4-R9}     ; restore registers
34     MOV    PC, LR           ; return
35
36 Cond1 ADD R4, R4, #1 ; case 'A': top++
37     B      Return
38 Cond2 ADD R5, R5, #1 ; case 'B': Bgrading++
39     B      Return
40 Cond3 ADD R6, R6, #1 ; case 'C': other++
41     B      Return
42 Cond4 ADD R6, R6, #1 ; case 'D': top++ (note: slides show R6; should be R4)
43     B      Return
44
45 ; Jump table (Base) --- one word per case, A=0, B=1, C=2, D=3
46 Base DCD Cond1
47     DCD Cond2
48     DCD Cond3
49     DCD Cond4

```

7.7 Loop Constructs

Loop constructs are distinguished by the position of their conditional test relative to the loop body:

Definition 7.7.1 (Pre-test Loop). A loop that evaluates its condition *before* executing the loop body. If the condition is false on the first check, the loop body is **never executed**. Examples: WHILE, FOR.

Definition 7.7.2 (Post-test Loop). A loop that evaluates its condition *after* executing the loop body. The loop body is always executed **at least once**. Example: DO-WHILE.

7.7.1 WHILE Implementation

The WHILE loop is a pre-test construct. The condition is reversed (as with the IF construct) to skip the unconditional jump, and an unconditional branch at the end of the body returns to the top of the loop:

```
1 ; C: while (VarX > 0) { Loop segment }
2
3 Back CMP "VarX", #0 ; load VarX into a register first
4     BLE Exit        ; reversed condition: exit if VarX <= 0
5     ...             ; loop body
6     B   Back         ; unconditional jump back to test
7 Exit :
```

7.7.2 DO-WHILE Implementation

The DO-WHILE loop is a post-test construct. The condition appears *after* the loop body, so no unconditional branch is needed at the top:

```
1 ; C: do { Loop segment } while (VarX > 0);
2
3 Back ...             ; loop body (always executes at least once)
4     CMP "VarX", #0
5     BGT Back         ; loop back if condition still true
6     :               ; exit
```

Key Result: DO-WHILE is More Efficient than WHILE

A post-test (DO-WHILE) loop requires *one fewer instruction* per iteration than the equivalent pre-test (WHILE) loop, because the unconditional back-branch is eliminated — the conditional branch at the bottom serves as both the loop test and the back-jump.

7.7.3 FOR Implementation

The FOR loop is conceptually a pre-test loop (condition checked before first iteration). However, optimising compilers often convert it into a post-test loop using a **decrement and test for zero** pattern, provided the loop counter is not used inside the loop body:

```
1 ; C: for (N = 0; N < 5; N++) { Loop segment }
2
3 ; --- Pre-test (direct translation) ---
4     MOV R0, #0
5 Back CMP R0, #5
6     BGE Exit        ; exit if N >= 5
```

```

7      ...           ; loop body
8      ADD R0, R0, #1 ; N++
9      B     Back
10 Exit :
11
12 ; --- Post-test (optimised: count down, test for zero) ---
13 MOV R0, #5        ; load count = N (counting down)
14 Back ...          ; loop body
15 SUBS R0, R0, #1    ; decrement, set flags
16 BNE Back          ; loop while counter != 0

```

Remark 7.7.3. The decrement-and-test-for-zero optimisation is valid only when the loop counter value is *not referenced* inside the loop body. When N is used (e.g. to index an array), the pre-test version with an incrementing counter must be used instead.

7.7.4 Example: Summation 1 to N

Computing $\sum_{i=0}^N i$ demonstrates the pre-test and post-test implementations side by side:

```

1 ; C: Sum = 0; for (i = 0; i <= N; i++) Sum += i;
2
3 ; --- Pre-test (7 instructions per loop iteration) ---
4     MOV R2, #0        ; i = 0
5     MOV R0, #0        ; Sum = 0
6 Loop CMP R2, #N
7     BGT END
8     ADD R0, R0, R2     ; Sum += i
9     ADD R2, R2, #1     ; i++
10    B     Loop
11 END  STR R0, ["sum"]
12
13 ; --- Post-test (5 instructions per loop iteration) ---
14     MOV R2, #0        ; i = 0
15     MOV R0, #0        ; Sum = 0
16 Loop ADD R0, R0, R2    ; Sum += i
17     ADD R2, R2, #1     ; i++
18     CMP R2, #5         ; check i <= N (N=5 here)
19     BLE Loop
20     STR R0, ["sum"]

```

7.8 Chapter Summary

- The **IF** construct is implemented by reversing the test condition to eliminate an extra unconditional branch.
- The **IF-ELSE** construct uses one conditional and one unconditional branch. Reversing the condition helps only when the ELSE body is more frequently executed.
- **Conditional execution** in 32-bit ARM allows any instruction to carry a condition suffix, eliminating branches and preventing pipeline flushes — applicable to both IF and IF-ELSE bodies.
- For **compound AND**: place the *least likely* condition leftmost for short-circuit efficiency. For **compound OR**: place the *most likely* condition leftmost.
- **Branchless logic** exploits arithmetic/bitwise relationships to compute Boolean outcomes without any branch instruction.

- **SWITCH** constructs with consecutive narrow-range values use a **jump table** for $O(1)$ dispatch. Wide-range random values use a **forked if-else-if** (binary partition) to reduce the average number of comparisons.
- **Post-test loops** (DO-WHILE) are more efficient than pre-test loops (WHILE, FOR) because they require one fewer unconditional branch per iteration.
- Optimising compilers may convert pre-test FOR loops into post-test decrement-and-test loops when the counter is not used inside the body.

Chapter 8

Instruction Encoding and ISA

8.1 Introduction

For a CPU to execute assembly instructions, each unique instruction must be encoded as a unique binary pattern. In a 32-bit ARM system, instructions are encoded within a 32-bit word. The instruction word must specify not only the operation type (e.g., ADD, SUB, MOV) but also the operands, conditions for conditional execution, and various control flags. This chapter examines the ARM instruction encoding format and explores broader issues in instruction set architecture (ISA) design.

Learning Objectives: Chapter 8

- Describe the instruction format of ARM instruction-set architecture (ISA)
- Understand the differences between fixed-length and variable-length ISA
- Recognise the implications of ISA design choices on performance, code density, and hardware complexity

8.2 Instruction Encoding Fundamentals

8.2.1 What is Encoded in an Instruction?

An ARM 32-bit instruction word encodes multiple pieces of information:

- **Instruction type** — identifies the operation (ADD, SUB, LDR, etc.)
- **Operand(s)** — specifies source and destination registers or immediate values
- **Condition for conditional execution** — determines when the instruction executes
- **Other flags** — such as whether to update condition flags (S bit), shift operations, etc.

The 32-bit instruction word is divided into fields, with bit positions allocated to each component. The challenge in ISA design is to fit all necessary information into a fixed number of bits while supporting a rich set of instructions and addressing modes.

8.3 ARM Instruction Format

ARM uses a **fixed-length** instruction format: every instruction is exactly 32 bits (in standard ARM mode). This simplifies instruction fetch and decode but places constraints on what can be encoded in a single instruction. We examine the encoding of different instruction categories below.

8.3.1 Arithmetic and Logic Instructions

Arithmetic and logic instructions (ADD, ADC, SUB, SBC, RSB, AND, EOR, RSC, ORR, BIC) follow one of three encoding patterns, depending on the second operand.

Register operand with immediate shift XXXCCS Rd, Rs1, Rs2, {shft #}

31–28	27–24	23–21	20	19–16	15–12	11–7	6–5	4	3–0
Cond	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

- **Cond** (bits 31–28): condition code
- **Inst. type** (bits 23–21): operation type
- **S** (bit 20): update condition flags
- **Rs1** (bits 19–16): first source register
- **Rd** (bits 15–12): destination register
- **Shift size** (bits 11–7): shift amount
- **shft** (bits 6–5): shift type (LSL, LSR, ASR, ROR)
- **Rs2** (bits 3–0): second source register

Register operand with register-specified shift XXXCCS Rd, Rs1, Rs2, {shft Rshift}

31–28	27–24	23–21	20	19–16	15–12	11–8	7	6–5	4	3–0
Cond	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

- **Rshift** (bits 11–8): register holding shift amount
- Bit 4 = 1 distinguishes register-specified shift

Immediate operand XXXCCS Rd, Rs1, #immediate (rotated)

31–28	27–24	23–21	20	19–16	15–12	11–8	7–0
Cond	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

- Bits 27–24 = 001 indicate immediate operand
- **#rot** (bits 11–8): rotation value (multiply by 2 to get actual rotation)
- **8-bit immediate** (bits 7–0): the immediate value, which can be rotated right by $2 \times \text{\#rot}$ positions to produce a 32-bit constant

Example 8.3.1 (Arithmetic Instruction Encodings). Consider the following ARM instructions and their encodings:

1. ADD R0, R1, R2

1110	000	0100	0	0001	0000	00000	00	0	0010
------	-----	------	---	------	------	-------	----	---	------

(Condition: always, ADD opcode = 0100, S=0, no shift)

2. ADD R0, R1, R2, LSL #5

1110	000	0100	0	0001	0000	00101	00	0	0010
------	-----	------	---	------	------	-------	----	---	------

(Same as above, but shift size = 5, shift type = LSL = 00)

3. SUBS R4, R3, R2, LSR R5

1110	000	0010	1	0011	0100	0101	0	01	1	0010
------	-----	------	---	------	------	------	---	----	---	------

(Condition: always, SUB opcode = 0010, S=1, Rshift = R5, LSR = 01, bit 4 = 1)

4. RSBEQS R5, R3, #20

0000	001	0011	1	0011	0101	0000	00010100
------	-----	------	---	------	------	------	----------

(Condition: EQ = 0000, immediate operand: bits 27–24 = 001, RSB opcode = 0011, S=1, immediate value = 20)

8.3.2 Comparison Instructions

Comparison instructions (TST, TEQ, CMP, CMN) always set condition flags but do not write a destination register. Their encoding is similar to arithmetic instructions, but the destination register field (Rd) is set to 0000, and the S bit is implicitly 1.

XXXCC Rs1, Rs2, {shft #}

31–28	27–24	23–21	20	19–16	15–12	11–7	6–5	4	3–0
Cond	000	Inst. type	1	Rs1	0000	Shift size	shft	0	Rs2

XXXCC Rs1, #immediate (rotated)

31–28	27–24	23–21	20	19–16	15–12	11–8	7–0
Cond	001	Inst. type	1	Rs1	0000	#rot	8-bit immediate

8.3.3 MOV Instructions

MOV and MVN instructions move data from a source to a destination register, optionally applying a shift. They do not use a first source register, so Rs1 is set to 0000.

XXXCCS Rd, Rs, {shft #}

31–28	27–24	23–21	20	19–16	15–12	11–7	6–5	4	3–0
Cond	000	Inst. type	S	0000	Rd	Shift size	shft	0	Rs

XXXCC Rd, #immediate (rotated)

31–28	27–24	23–21	20	19–16	15–12	11–8	7–0
Cond	001	Inst. type	S	0000	Rd	#rot	8-bit immediate

Example 8.3.2 (MOV Instruction Encodings). 1. MOV R0, R5

1110	000	1101	0	0000	0000	00000	00	0	0101
------	-----	------	---	------	------	-------	----	---	------

(MOV opcode = 1101, no shift, Rs1 = 0000)

2. MOVNE R1, R3, LSL R2

0001	000	1101	0	0000	0001	0010	0	00	1	0011
------	-----	------	---	------	------	------	---	----	---	------

(Condition: NE = 0001, shift register R2, LSL = 00, bit 4 = 1)

3. MOV R1, #0x40000000

1110	001	1101	0	0000	0001	0100	01000000
------	-----	------	---	------	------	------	----------

(Immediate operand: bits 27–24 = 001, rotation = 4, immediate = 0x40)

8.3.4 Shift and Rotate Instructions

The shift and rotate instructions (LSL, LSR, ASR, ROR, RRX) do not have dedicated instruction opcodes. Instead, they are encoded as MOV instructions with an appropriate shift operation applied to the source register.

Key Insight: Shift Instructions as MOV

In ARM, LSL R1, R2, #5 is equivalent to MOV R1, R2, LSL #5. The shift operations are encoded in the shift field (bits 6–5) of the MOV instruction:

Code	Shift type
00	LSL
01	LSR
10	ASR
11	ROR/RRX

8.3.5 Branch Instructions

Branch instructions (B, BL) use a large offset field to specify the target address. The offset is a signed 24-bit value, which is shifted left by 2 bits (multiplied by 4) to give a byte offset, allowing a branch range of ± 32 MB.

B(L)CC Offset

31–28	27–25	24	23–0
Condition	101	L	Offset

- **L** (bit 24): link bit (1 for BL, 0 for B)
- **Offset** (bits 23–0): signed 24-bit offset (in words, shifted left by 2 to get byte offset)

The assembler calculates the offset automatically from the target label.

8.3.6 Load/Store Instructions

Load and store instructions (LDR, LDRB, STR, STRB) encode the base register, offset, and various control bits for indexing modes and data size.

LDR/STR{B}CC Rs, [RD, offset pre or post]W

Register offset:

31–28	27–26	25	24	23	22	21	20	19–16	15–12	11–7	6–5	4	3–0
Cond	01	0	P	U	B	W	L	Rs	Rd	Shift size	shft	0	Rs

Immediate offset:

31–28	27–26	25	24	23	22	21	20	19–16	15–12	11–0
Cond	01	1	P	U	B	W	L	Rs	Rd	Immediate value

- **P** (bit 24): pre-indexing (1) or post-indexing (0)
- **U** (bit 23): add offset (1) or subtract offset (0)
- **B** (bit 22): byte access (1) or word access (0)
- **W** (bit 21): write-back (1) or no write-back (0)
- **L** (bit 20): load (1) or store (0)

Example 8.3.3 (Load/Store Encodings). 1. LDR R0, [R2, #4]

1110	01	1	1	1	0	0	1	0010	0000	000000000100
------	----	---	---	---	---	---	---	------	------	--------------

(Load word, pre-indexed, add offset, no write-back)

2. STRB R0, [R2, #-4]!

1110	01	1	1	0	1	1	0	0010	0000	000000000100
------	----	---	---	---	---	---	---	------	------	--------------

(Store byte, pre-indexed, subtract offset, write-back)

8.3.7 Load/Store Multiple Instructions

Load and store multiple instructions (LDM, STM) transfer multiple registers to or from memory in a single instruction. The register list is encoded as a 16-bit bitmask.

LDM/STMF_{XCC} R_sW, {register list}

31–28	27–25	24	23	22	21	20	19–16	15–0
Cond	100	P	U	^	W	L	Rs	Register list

- **P** (bit 24): increment before (1) or after (0)
- **U** (bit 23): increment (1) or decrement (0)
- **^** (bit 22): don't care (used for user-mode register transfer in privileged modes)
- **W** (bit 21): write-back (1) or no write-back (0)
- **L** (bit 20): load (1) or store (0)
- **Register list** (bits 15–0): each bit corresponds to a register (R0–R15)

The combination of P and U bits determines the stack addressing mode:

P	U	Instruction
1	0	STMFD (full descending)
0	1	LDMFD (full descending)
0	0	STMFA (full ascending)
1	1	LDMFA (full ascending)

Example 8.3.4 (Load/Store Multiple Encoding). STMFD SP!, {R0--R6}

1110	100	1	0	^	1	0	1101	0000000001111111
------	-----	---	---	---	---	---	------	------------------

(SP = R13 = 1101, P=1, U=0 for STMFD, W=1 for write-back, L=0 for store, register list bits 0–6 set for R0–R6)

8.4 Instruction Type and Condition Field Encoding

8.4.1 Instruction Type Field

The instruction type field uses 4 bits (bits 23–21 for data processing instructions), allowing up to 16 different operations. The ARM ISA assigns these codes as follows:

Code	Instruction	Code	Instruction
0000	AND	1000	TST
0001	EOR	1001	TEQ
0010	SUB	1010	CMP
0011	RSB	1011	CMN
0100	ADD	1100	ORR
0101	ADC	1101	MOV
0110	SBC	1110	BIC
0111	RSC	1111	MVN

Similarly, 4 bits are allocated for source and destination registers, allowing up to 16 registers (R0–R15).

8.4.2 Condition Field

The condition field (bits 31–28) allows every ARM instruction to be conditionally executed based on the current state of the condition flags (N, Z, C, V). This is a unique feature of the ARM architecture.

Code	Condition	Flags
0000	EQ (equal)	Z = 1
0001	NE (not equal)	Z = 0
0010	CS/HS (carry set / unsigned $\geq$)	C = 1
0011	CC/LO (carry clear / unsigned $<$)	C = 0
0100	MI (negative)	N = 1
0101	PL (positive or zero)	N = 0
0110	VS (overflow set)	V = 1
0111	VC (overflow clear)	V = 0
1000	HI (unsigned higher)	C = 1 and Z = 0
1001	LS (unsigned lower or same)	C = 0 or Z = 1
1010	GE (signed $\geq$)	N = V
1011	LT (signed $<$)	N $\neq$ V
1100	GT (signed $>$)	Z = 0 and N = V
1101	LE (signed $\leq$)	Z = 1 and N $\neq$ V
1110	AL (always)	—
1111	NV (never / reserved)	—

Key Feature: Conditional Execution

Every ARM instruction can be conditionally executed by appending a condition suffix. This eliminates the need for many branch instructions and helps maintain pipeline efficiency. The condition field occupies 4 bits of every instruction word.

8.5 Fixed-Length vs. Variable-Length ISA

Instruction set architectures can be broadly classified into two categories based on instruction length: **fixed-length** and **variable-length**.

8.5.1 Fixed-Length ISA

ARM uses a fixed-length instruction format: all instructions are exactly 32 bits (in standard ARM mode). This design choice has several implications:

- **Simplicity of fetch and decode:** The CPU always knows exactly how many bytes to fetch for each instruction. Instruction boundaries are trivially aligned on 4-byte boundaries.
- **Pipelining efficiency:** Fixed instruction length simplifies the design of the fetch and decode stages of the pipeline.
- **Operand constraints:** Operands must be registers or small immediate values. Large constants cannot be encoded directly and must be loaded from memory or constructed using multiple instructions.
- **Code density:** Fixed-length instructions may result in lower code density (more memory required to store a program) compared to variable-length ISAs.

8.5.2 Variable-Length ISA

Variable-length ISAs, such as Intel x86, allow instructions of different lengths. For example:

- **Intel 80x86:** Instructions vary from 1 to 17 bytes.

- **Digital VAX:** Instructions vary from 1 to 54 bytes.

Variable-length ISAs offer several advantages and disadvantages:

- **Flexibility:** Instructions can encode complex operations with multiple operands and addressing modes in a single instruction.
- **Code density:** More compact encoding for simple operations; less memory required to store programs.
- **Complexity:** Multi-step fetch and decode; instruction boundaries are not aligned; more complex hardware.
- **Performance trade-offs:** Decoding variable-length instructions is slower and more complex, potentially offsetting the benefits of higher code density.

Example 8.5.1 (Intel x86 Instruction Format). The Intel x86 architecture uses a highly complex variable-length instruction format. An instruction consists of:

- Optional instruction prefixes (1 byte each)
- 1-, 2-, or 3-byte opcode
- Optional ModR/M byte (specifies operands)
- Optional SIB (Scale-Index-Base) byte (for complex addressing)
- Optional displacement (1, 2, or 4 bytes)
- Optional immediate data (1, 2, or 4 bytes)

This flexibility allows for compact code but requires complex decoding hardware and multi-cycle instruction fetch.

Key Trade-off: Fixed vs. Variable Length

- **Fixed-length** (ARM): Simple hardware, easier pipelining, but lower code density and limited immediate operands.
- **Variable-length** (x86): Higher code density, complex operations in single instruction, but complex hardware and slower decode.

8.6 Design Considerations in ISA

8.6.1 Using Registers Efficiently

Operations using registers are significantly faster than those involving memory due to the physical proximity of registers to the ALU and the shorter data paths.

Memory Hierarchy Access Times (Approximate)

Memory Level	Access Time
Register	< 1 cycle
L1 Cache	1–4 cycles
L2 Cache	10–20 cycles
DRAM	100–300 cycles

To maximise performance, programs should keep frequently used data in registers. This is one reason why modern ISAs provide a reasonably large register file (e.g., ARM has 16 general-purpose registers).

Key Performance Principle: Minimise Memory Access

Keep as much of your computation within registers as possible. Each memory access incurs a significant performance penalty compared to register operations.

8.6.2 Implications of Register Count

Increasing the number of registers has both benefits and costs:

Benefits:

- More opportunities for keeping data in registers, reducing memory traffic.
- Better support for complex algorithms that require many temporary variables.

Costs:

- **Silicon area:** More registers consume more chip area.
- **Instruction encoding:** More registers require more bits to encode register operands. For example, encoding 16 registers requires 4 bits per operand, while encoding 32 registers requires 5 bits. In a fixed-length ISA, this reduces the bits available for other fields.
- **Routing complexity:** More registers increase the complexity of the multiplexers and data paths connecting registers to the ALU and memory.
- **Context-switching overhead:** More registers mean more state to save and restore during context switches.

Example 8.6.1 (Register Encoding Overhead). Consider a 32-bit fixed-length instruction for a 3-operand operation (e.g., ADD Rd, Rs1, Rs2):

- With 16 registers: $3 \times 4 = 12$ bits for operands.
- With 32 registers: $3 \times 5 = 15$ bits for operands.

The extra 3 bits consumed by register encoding reduces the space available for other fields such as immediate values or opcodes.

8.6.3 Orthogonality of ISA

Definition 8.6.2 (Orthogonality). An ISA is **orthogonal** if every instruction can use every available addressing mode, and there are no restrictions on which registers can be used with which instructions. In a truly orthogonal ISA, instruction type and operand specification are independent.

Complete orthogonality is difficult to achieve in a fixed-length ISA because encoding all possible combinations of operations and addressing modes requires a large number of bit patterns. Increasing instruction length to achieve orthogonality would increase program size and memory requirements.

Remark 8.6.3. Most real-world ISAs are not fully orthogonal. For example, ARM restricts certain operations (e.g., multiplication) to specific register combinations, and some instructions (e.g., load/store) cannot use certain addressing modes available to arithmetic instructions.

Warning: Non-Orthogonal Restrictions

When writing assembly code, be aware of instruction-specific restrictions. For instance, not all ARM instructions support shifted register operands, and some instructions implicitly use specific registers (e.g., MUL results always go to specific register pairs in older ARM versions). Consult the architecture reference manual to avoid invalid instruction encodings.

8.7 Why Good ISA Design Matters

Good instruction set architecture design can yield significant benefits across multiple dimensions:

- **Execution performance:** Well-designed instruction encoding allows for efficient pipelining and out-of-order execution. Instructions that map naturally to hardware operations execute faster.
- **Code density:** Efficient encoding reduces the memory footprint of programs. This is especially important for embedded systems with limited memory.
- **Power consumption:** Fewer memory accesses and simpler instruction decode reduce dynamic power consumption. Fixed-length instructions generally require less decoding logic than variable-length instructions.
- **Manufacturing cost:** Simpler instruction decode hardware reduces chip area and manufacturing cost. Fixed-length ISAs are easier to implement in hardware.
- **Programmer productivity:** A well-designed ISA with clear orthogonality and minimal special cases makes it easier for programmers (and compilers) to write efficient code.
- **Compiler optimisation:** ISAs that map cleanly to high-level language constructs enable better compiler optimisations and more predictable performance.

Key Insight: ISA Design is a Multi-Objective Optimisation Problem

ISA designers must balance competing objectives: performance, code density, hardware complexity, power consumption, and ease of use. There is no single “best” ISA; the optimal design depends on the intended application domain (e.g., embedded systems, high-performance computing, general-purpose processors).

8.7.1 Custom Processor Design in Industry

Designing a processor (or co-processor) is no longer exclusively the domain of traditional hardware companies. With the rise of open ISAs (e.g., RISC-V), domain-specific accelerators, and hardware description languages (HDLs), software companies and startups are increasingly designing custom processors tailored to specific workloads:

- **Google:** Designed the Tensor Processing Unit (TPU), a custom ASIC optimised for machine learning inference and training. The TPU uses a specialised ISA focused on matrix operations and achieves significantly higher performance per watt than general-purpose CPUs for neural network workloads.
- **Facebook (Meta):** Developing custom ASICs for video processing and AI inference.
- **Amazon:** Designed the Graviton series of ARM-based server processors and the Inferentia/Trainium AI accelerators.

- **Tesla:** Designed custom AI chips for autonomous driving.
- **NVIDIA:** While traditionally a hardware company, NVIDIA's CUDA ISA and Tensor Cores demonstrate the value of domain-specific instruction set extensions.

This trend highlights the importance of understanding ISA design principles: even if you do not design processors professionally, knowledge of instruction encoding and ISA trade-offs is increasingly relevant in software engineering and system design.

8.8 Chapter Summary

- ARM uses a **fixed-length** 32-bit instruction format, encoding instruction type, operands, condition codes, and control flags within each instruction word.
- **Arithmetic and logic instructions** encode operation type, source and destination registers, and optional shift operations. Immediate operands use an 8-bit value with a 4-bit rotation field to construct 32-bit constants.
- **Comparison instructions** (CMP, CMN, TST, TEQ) always set condition flags and do not write a destination register.
- **MOV instructions** transfer data between registers, optionally applying a shift. Shift and rotate instructions (LSL, LSR, ASR, ROR, RRX) are encoded as MOV with shift.
- **Branch instructions** use a 24-bit signed offset to specify the target address, providing a branch range of ± 32 MB.
- **Load/store instructions** encode base register, offset, addressing mode (pre/post-indexing), and data size (byte/word). Load/store multiple instructions use a 16-bit register bitmask.
- The **condition field** (bits 31–28) allows every ARM instruction to be conditionally executed based on the state of the condition flags, reducing the need for branch instructions.
- **Fixed-length ISAs** (e.g., ARM) simplify instruction fetch and decode, enable efficient pipelining, but constrain operand encoding and may reduce code density.
- **Variable-length ISAs** (e.g., Intel x86) allow more flexible and compact encoding but require complex multi-cycle decoding hardware.
- **Register usage** is critical for performance: operations on registers are orders of magnitude faster than memory accesses. Keeping frequently used data in registers minimises memory traffic.
- Increasing the number of registers has trade-offs: more registers reduce memory traffic but increase instruction encoding overhead, silicon area, and routing complexity.
- **ISA orthogonality** (every instruction can use every addressing mode) is difficult to achieve in fixed-length ISAs due to encoding constraints. Most real-world ISAs have non-orthogonal restrictions.
- Good ISA design balances performance, code density, power consumption, hardware complexity, and programmer productivity. The optimal ISA depends on the target application domain.
- Custom processor design is increasingly common outside traditional hardware companies, with software companies designing domain-specific accelerators (e.g., Google TPU). Understanding ISA principles is valuable even for software engineers.